

CS

紧致简单的双峰格签名算法

李帅钢^{1,2}, 白亮^{1,2}, 郑中翔³, 黄可腾^{1,2}, 丁颜^{1,2}, 马永彪¹, 王雨辰¹

¹ 兴唐通信科技有限公司

² 抗量子计算密码北京市重点实验室

³ 中国传媒大学

2026 年 6 月 25 日

目录

1 简介	2
1.1 设计原理	2
1.2 优势与不足	3
1.2.1 优势	4
1.2.2 不足	4
2 基础知识	4
2.1 符号与分布	4
2.2 NTT 域表示	5
2.3 随机环元素的生成	6
2.4 挑战多项式的生成	6
2.5 高低比特与提示	6
2.6 rANS 编码	8
2.7 <i>Reject</i> 算法	8
3 签名	10
3.1 拒绝采样算法	10
3.2 简化 CS 签名算法	13
3.3 完整 CS 签名算法	15
3.4 正确性与签名重复次数	15
3.5 算法参数及选择依据	19
4 算法安全性	20
4.1 密码学定义	20
4.2 签名算法安全性	22
4.3 实际安全强度	25
4.4 其他安全特性	26
5 算法性能与算法特点	27
5.1 算法性能	27
5.2 算法特点	27
参考文献	28
附录	30
A 附录 A 中心多项分布的性质	30
B 附录 B 拒绝采样算法的相关引理	31

1 简介

根据签名生成机制的不同, Fiat-Shamir (FS) 类格签名算法可以分为单峰 FS 算法和双峰 FS 算法两大类。单峰 FS 算法 (如 NIST 标准化的 ML-DSA 算法 [LDK+22, FIPS204]) 易实现全流程无浮点运算, 具有良好的抗侧信道能力, 但其公钥和签名尺寸较大; 双峰 FS 算法 (如韩国标准化的 HAETAE 算法 [CHE+24, CHE+23]) 能实现更紧致的公钥和签名尺寸, 但常涉及浮点运算, 侧信道防护难度更大。

为同时突破这两条路线的瓶颈, 我们设计了签名算法 CS。C 是寸 (Cun) 的首字母, 也是 Compact 的首字母, 代表其在公钥与签名尺寸上的紧致性, S 是算 (Suan) 的首字母, 也是 Simple 的首字母, 代表其完全依赖简单的整数运算。CS 算法全流程无浮点运算, 同时具备易抗侧信道和紧致两大算法特性。

CS 算法是一个基于模格上容错学习问题 (MLWE) 与短整数求解问题 (MSIS) 的双峰 FS 格签名算法。该算法使用简单易采样的中心多项分布作为掩码分布, 通过整数运算使得拒绝采样算法无浮点运算, 从而具备良好的抗侧信道能力。CS 还兼容有效的公钥压缩技术, 并使用更大的挑战多项式空间, 从而实现了紧致的公钥和签名尺寸。此外, CS 提出更灵活的迭代双峰设计来提升算法效率。总体来说,

CS = 模格 + 中心多项分布 + 有效公钥压缩 + 灵活迭代双峰 $\Rightarrow$ 紧致尺寸 + 全流程无浮点

1.1 设计原理

FS 类格签名算法 [Lyu09, Lyu12, BG14] 是 Schnorr 签名算法 [Sch91] 的变种, NIST 标准化的 ML-DSA 算法 [LDK+22, FIPS204] 以及韩国标准化的 HAETAE 算法 [CHE+24, CHE+23] 都是基于格的 FS 类签名算法。最初的 FS 类格签名算法的公钥通常由一个矩阵 $\mathbf{A}$ 和一个向量 $\mathbf{b}$ 组成, 私钥是一个向量 $\mathbf{s}$, 满足 $\mathbf{As} = \mathbf{b} \pmod{q}$ 。给定一个消息 μ , 签名算法首先生成一个掩码向量 $\mathbf{y}$, 然后计算承诺 $\mathbf{w} = \mathbf{Ay}$ 、挑战多项式 $c = H(\mathbf{w}, \mu)$ 以及响应多项式 $\mathbf{z} = \mathbf{y} + c\mathbf{s}$, 最后通过拒绝采样算法以一定的概率输出签名 $(\mathbf{z}, c)$, 否则输出 $\perp$ 。给定一个消息 μ 及其签名 $(\mathbf{z}, c)$, 验证者计算承诺多项式 $\mathbf{w}' = \mathbf{Az} - c\mathbf{b} = \mathbf{A}(\mathbf{y} + c\mathbf{s}) - c\mathbf{b} = \mathbf{Ay} = \mathbf{w} \pmod{q}$, 若 $\mathbf{z}$ 较小且 $c = H(\mathbf{w}', \mu)$ 则验证通过, 否则验证失败。双峰 FS 类格签名算法的公私钥通常满足 $\mathbf{As} = q\mathbf{j} \pmod{2q}$ ($\mathbf{j}$ 是一个单位向量), 签名生成算法除了以一定的概率输出 $\mathbf{z} = \mathbf{y} + c\mathbf{s}$ 和 $\perp$, 还一定的概率输出 $\mathbf{z} = \mathbf{y} - c\mathbf{s}$ 。

我们设计的 CS 算法的核心特点是全流程无浮点运算且公钥与签名尺寸之和较小。为了实现这两个目标, CS 算法在传统双峰 FS 类签名算法的基础上使用了如下关键设计:

双峰中心多项分布。这是实现 CS 算法全流程无浮点运算的关键一环。无浮点设计的关键在于两点: 一是掩码 $\mathbf{y}$ 的采样过程无浮点运算, 二是拒绝采样算法中的拒绝概率计算无浮点。常见的掩码 $\mathbf{y}$ 分布有多种, 包括: 高斯分布、超球上的均匀分布以及超立方体上的均匀分布等。高斯分布、超球上的均匀分布在采样的过程中均涉及浮点运算, 超立方体均匀分布的采样和拒绝概率计算过程均不涉及浮点运算, 但该分布不兼容双峰设计致使签名重复次数较高, 为了降低重复次数, 其参数设置较大致使其签名和公钥尺寸较大。为此, 我们使用双峰中心多项分布, 中心多项分布是中心二项分布的自然推广, 其采样算法简单且无浮点操作。此外, 中心多项分布的概率质量函数能通过两个整数之比精确表示以避免浮点运算, 这使得拒绝概率可以仅在整数运算下完成, 因此, 保证了 CS 算法全流程无浮点运算。但这使得拒绝概率计算涉及多个整数的连乘操作, 为

为了避免大整数参与乘法运算降低效率，我们使用整数截断来保证其高效性，并使用 Rényi 散度证明截断对算法安全性的影响可控。

有效的公钥压缩。现有的双峰 FS 类格签名算法通常和高效的公钥压缩机制不兼容，如 [DDLL13, WKH⁺25] 无公钥压缩，[CHE⁺24] 对公钥的压缩最多为 1 比特。CS 算法像 ML-DSA 算法一样，能对公钥进行有效的压缩，不仅降低了公钥尺寸，同时提升了算法的灵活性。

更大的挑战多项式空间。传统的双峰 FS 类格签名算法的挑战多项式的系数均采用 $\{0, 1\}$ 上的固定汉明重量，CS 算法则使用 $\{0, \pm 1\}$ 上的固定汉明重量。在同样挑战熵的情况下，CS 算法挑战多项式的固定汉明重量更小，一定程度上降低了签名的尺寸。

灵活的迭代双峰设计。2025 年 [Gar25] 提出了迭代双峰 FS 类格签名算法，迭代双峰的设计能有效降低签名算法的重复次数，从而提升签名效率。不同于 [Gar25] 的双峰高斯设计，CS 算法使用双峰中心多项分布，影响其签名算法效率的另一个关键因素是拒绝概率计算复杂度。使用 [Gar25] 的迭代双峰设计，能降低签名的重复次数，但签名的迭代次数增加导致拒绝概率计算复杂度增加。而采用无迭代的双峰设计 [DDLL13, CHE⁺24, WKH⁺25]，拒绝概率计算复杂度降低，但签名的重复次数增加。为了更好地平衡这两个因素，提升 CS 签名算法的整体效率，我们引入了一个参数 τ' 以更好地平衡签名重复次数与拒绝概率计算复杂度，从而提升签名算法的效率。

总之，在传统双峰 FS 格签名算法的基础上，我们使用简单易采样的双峰中心多项分布，通过整数运算实现相应的拒绝采样过程，使得 CS 算法全流程无浮点运算。此外，CS 算法使用有效的公钥压缩技术以及更大的挑战多项式空间来降低签名与公钥的尺寸。同时，我们提出灵活的迭代双峰设计来提升算法的效率。最终设计出了全流程无浮点运算的紧致的格签名算法 CS。

1.2 优势与不足

表 1 展示了 HAETAE 算法、ML-DSA 算法以及 CS 算法的尺寸情况，总体来看，CS 签名算法在公钥和签名尺寸方面具有优势，其公钥是三者中最小的，公钥与签名尺寸之和几乎也是最小的。表 2 展示了这三个算法的效率情况，Hash 函数使用的是 SM3，总体来看，CS 算法在密钥生成和签名验证方面表现出较好的效率，密钥生成是三者中最快速的，验证效率也很高。

表 1: 签名算法的尺寸对比（字节）

algorithms	public key	signature	sum	secret key
HAETAE-120	992	1474	2466	1408
ML-DSA-44	1312	2420	3732	2560
CS-128	976	1548	2524	1888
HAETAE-260	2080	2948	5028	2752
ML-DSA-87	2592	4627	7219	4896
CS-256	1760	3164	4924	3968
CS-512	4288	5975	10263	7808

表 2: 签名算法的效率对比 (kcycles)

algorithms	KeyGen	Sign	Verify
HAETAE-120	2722	11946	456
ML-DSA-44	996	2371	930
CS-128	503	4009	536
HAETAE-260	4892	22605	1160
ML-DSA-87	2885	5079	2789
CS-256	809	13083	1028
CS-512	2466	45269	3127

1.2.1 优势

1. **全流程无浮点运算，易抗侧信道。** HAETAE 等双峰 FS 签名算法常涉及浮点运算，不易进行侧信道防护，CS 算法和 ML-DSA 算法一样全流程无浮点运算，只涉及简单的整数运算，易进行侧信道防护。
2. **公钥与签名尺寸小。** 如表 1 所示在 128 和 256 安全级别，CS 算法的公钥尺寸是三者中最小的，分别约占 ML-DSA 的 75% 与 68%。CS 算法的签名尺寸与 HAETAE 相当，分别约占 ML-DSA 的 64% 与 69%。CS 算法的公钥与签名尺寸之和几乎也是三者中最小的，分别约占 ML-DSA 的 68% 与 69%。
3. **密钥生成和签名验证速度快。** 如表 2 所示，在 128 和 256 安全级别，CS 的密钥生成速度分别约为 HAETAE 算法的 5.4 与 6.0 倍，分别约为 ML-DSA 算法的 2.0 与 3.6 倍。CS 的签名验证速度分别约为 ML-DSA 的 1.7 与 2.7 倍，与 HAETAE 相当。因此，CS 算法在密钥生成和签名验证上均具有较优的效率。
4. **灵活的参数选择。** 除了底层的模格困难问题的灵活性，相较于传统的格签名算法，我们有更多的算法参数可以调节，通过优化参数配置，能更好地平衡安全性、尺寸和效率，以适配不同需求的应用场景。

1.2.2 不足

CS 算法的签名生成速度快于 HAETAE，但慢于 ML-DSA 算法。如表 2 所示，在 128 和 256 安全级别，CS 的签名生成速度分别约为 ML-DSA 的 59% 与 39%。但 CS 算法的签名生成速度仍优于 HAETAE 算法，分别约为 HAETAE 的 3.0 与 1.7 倍。

2 基础知识

2.1 符号与分布

本算法使用 $\lceil x \rceil, \lfloor x \rfloor$ 分别表示取有理数 x 的近似上整和下整， $\log$ 表示以 2 为底的对数， $LSB(x)$ 表示整数 x 的最低有效位。对于正整数 x, a ， $x \gg a$ 表示对 x 右移 a 位， $len(x)$ 表示

取 x 的比特数, $[x] = \{0, 1, \dots, x-1\}$, $\binom{x}{a}$ 表示从 x 个不同元素中随机选择 a 个元素的组合数。对于整数 x 和正整数 q , $y = x \bmod q$ 表示唯一的 $y \in [q]$ 使得 $(y-x) \bmod q = 0$, $y = x \bmod \pm q$ 表示存在唯一的 $y \in [-q/2, q/2)$ 使得 $(y-x) \bmod q = 0$ 。

默认情况下, 本算法用粗体小写字母表示列向量 (如: $\mathbf{y}$), 用粗体大写字母表示矩阵 (如: $\mathbf{A}$)。用 $(\mathbf{y}^\top, \mathbf{z}^\top)^\top$ 表示将列向量 $\mathbf{z}$ 级联在列向量 $\mathbf{y}$ 的下方, 用 $(\mathbf{y}|\mathbf{z})$ 表示将列向量 $\mathbf{z}$ 级联在列向量 $\mathbf{y}$ 的右边, 可以自然地将后者推广到向量和矩阵 (如: $(\mathbf{A}|\mathbf{B})$) 以及矩阵和矩阵 (如: $(\mathbf{A}|\mathbf{B})$)。

$\mathcal{R} = \mathbb{Z}[x]/(x^n + 1)$ 表示一个多项式环, 其中 n 是 2 的幂次, 定义 $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$, 其中 q, n 满足 $2n|(q-1)$ 以方便使用 NTT 加速多项式乘法, 本算法还定义 $\mathcal{R}_{2q} = \mathbb{Z}_{2q}[x]/(x^n + 1)$ 。此外, 定义 $\mathcal{R}_q^k, \mathcal{R}_q^{k \times \ell}$ 分别表示 $\mathcal{R}_q$ 上的 k 维向量以及 $\mathcal{R}_q$ 上的 $k \times \ell$ 维矩阵, 相应地 $\mathcal{R}_{2q}$ 也可以推广到其向量和矩阵形式。

整数变量 X 服从参数为 (a, b) 的中心多项分布 (CMD) $\mathcal{D}_{a,b}$, 则变量 X 的生成方法为:

$$\sum_{i=0}^{a-1} (x_i - y_i)$$

其中, $x_i \leftarrow [2^b], y_i \leftarrow [2^b]$ 。进一步, 定义 $\mathcal{D}_{a,b,r}$ 为分布 $\mathcal{D}_{a,b}$ 向右平移 r 个单位后获得。定义 $\overline{\mathcal{D}}_{a,b,r}^t$ 是将分布 $\mathcal{D}_{a,b,r}$ 截掉超出 $[-t, t]$ 部分形成的新分布 (该算法中 $t \leq a(2^b - 1) - |r|$)。双峰中心多项分布 $\mathcal{D}_{a,b,\pm r}$ 是以 $\frac{1}{2}$ 的概率取自于 $\mathcal{D}_{a,b,r}$ 且以 $\frac{1}{2}$ 的概率取自于 $\mathcal{D}_{a,b,-r}$, 相应的 $\overline{\mathcal{D}}_{a,b,\pm r}^t$ 是将分布 $\mathcal{D}_{a,b,\pm r}$ 截掉超出 $[-t, t]$ 部分形成的新分布。中心多项分布及其相关分布可推广到向量、多项式以及多项式向量。 $\mathbf{x} \sim \mathcal{D}_{a,b}^k$ 表示 k 维向量 $\mathbf{x}$ 的每个分量独立且 $\mathbf{x}_i \sim \mathcal{D}_{a,b}$, $x \sim \mathcal{D}_{a,b}^{\mathcal{R}}$ 表示多项式 $x \in \mathcal{R}$ 的系数独立取自于分布 $\mathcal{D}_{a,b}$, $\mathbf{x} \sim \mathcal{D}_{a,b}^{\mathcal{R}^k}$ 表示多项式向量 $\mathbf{x} \in \mathcal{R}^k$ 的每个多项式 $\mathbf{x}_i$ 的每个系数独立取自于分布 $\mathcal{D}_{a,b}$, 在不引起混淆的情况下, 将上述分布均简记为 $\mathcal{D}_{a,b}$ 。同样地, $\mathcal{D}_{a,b,r}, \overline{\mathcal{D}}_{a,b,r}^t, \mathcal{D}_{a,b,\pm r}, \overline{\mathcal{D}}_{a,b,\pm r}^t$ 也可以推广到向量、多项式以及多项式向量。此外, $\mathcal{S}_\eta$ 表示 $[-\eta, \eta]$ 上的整数均匀分布并将 η_1 简记为 $\mathcal{S}$, $\mathcal{S}_\eta$ 也可推广到向量、多项式以及多项式向量。

对于一个分布 $\mathcal{D}$, $x \leftarrow \mathcal{D}$ 表示 x 从分布 $\mathcal{D}$ 中均匀随机选取, 多项式 $x \leftarrow \mathcal{D}$ 表示多项式 $x \in \mathcal{R}$ 的系数从 $\mathcal{D}$ 中随机选取的, 多项式向量 $\mathbf{x} \leftarrow \mathcal{D}^k$ 表示 $\mathbf{x}$ 的每个多项式分量 $\mathbf{x}_i$ 的每个系数均从 $\mathcal{D}$ 中随机选取的。此外, 定义 $\mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}$ 表示 $k \times \ell$ 维多项式矩阵 $\mathbf{A}$ 从集合 $\mathcal{R}_q^{k \times \ell}$ 中随机选择。

2.2 NTT 域表示

CS 签名算法涉及到的多项式环为 $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$ 和 $\mathcal{R}_{2q} = \mathbb{Z}_{2q}[x]/(x^n + 1)$, n 为 2 的幂次, q 是满足 $q \equiv 1 \pmod{2n}$ 的素数。乘法群 $\mathbb{Z}_q^*$ 中存在阶为 $2n$ 的本原单位根。然而, 模数 $2q$ 为合数, 直接在模 $2q$ 下执行数论变换 (NTT) 是不可行的。为此, 需要先映射到 $\mathbb{Z}_q$ 下使用 NTT 计算, 再通过剩余定理恢复到 $\mathbb{Z}_{2q}$ 。

由于 $q \equiv 1 \pmod{2n}$, 乘法群 $\mathbb{Z}_q^*$ 中存在阶为 $2n$ 的本原单位根 ζ 。为了优化底层运算结构, 我们选择不将多项式完全分解为一次因式, 而是将其分解为 $n/2$ 个二次因式的乘积。对 $i = 0, 1, \dots, n/2 - 1$, 我们将共轭根 ζ^{2i+1} 与 $\zeta^{-(2i+1)}$ 配对, 从而分圆多项式 $x^n + 1$ 在 $\mathbb{Z}_q$ 上可分解为:

$$x^n + 1 = \prod_{i=0}^{n/2-1} (x - \zeta^{2i+1})(x - \zeta^{-(2i+1)}) = \prod_{i=0}^{n/2-1} (x^2 - \eta_i x + 1),$$

其中 $\eta_i = \zeta^{2i+1} + \zeta^{-(2i+1)} \pmod{q}$ 。

由中国剩余定理 (CRT)，环 $\mathcal{R}_q$ 同构于 $n/2$ 个二次多项式商环的直积：

$$\mathcal{R}_q \cong \prod_{i=0}^{n/2-1} \mathbb{Z}_q[x]/(x^2 - \eta_i x + 1).$$

在该表示下，多项式 $a \in \mathcal{R}_q$ 被同构映射为 $n/2$ 个一次多项式。具体地，记其 NTT 表示为这 $n/2$ 个余式构成的向量：

$$\hat{a} = \text{NTT}(a) = (\hat{a}_0(x), \hat{a}_1(x), \dots, \hat{a}_{n/2-1}(x)),$$

其中，每个分量 $\hat{a}_i(x) = a(x) \bmod (x^2 - \eta_i x + 1)$ ，且形如 $\hat{a}_i(x) = c_{i,0} + c_{i,1}x$ 。

在该频域表示下，两个多项式 $a, b \in \mathcal{R}_q$ 的乘法，转化为对应 $n/2$ 个分量上的逐点多项式模乘：

$$a \cdot b = \text{NTT}^{-1}(\text{NTT}(a) \circ \text{NTT}(b)),$$

其中 $\circ$ 表示对应的余式分量 $\hat{a}_i(x)$ 与 $\hat{b}_i(x)$ 在模 $x^2 - \eta_i x + 1$ 下的基底多项式乘法 (Basecase Multiplication)， NTT^{-1} 为逆 NTT。

2.3 随机环元素的生成

CS 算法中的公钥矩阵、私钥向量以及签名掩码均从短种子通过哈希函数确定性地生成。下面简要说明这三个扩展函数。

- $\text{ExpandA}(\rho_0)$ ：输入为短种子 ρ_0 ，输出为公钥矩阵 $\mathbf{A}' \in \mathcal{R}_q^{k \times \ell}$ 的 NTT 形式。其实现方式是以 ρ_0 为根种子，为矩阵的每个元素派生子种子，再通过拒绝采样生成 $\mathcal{R}_q$ 上的均匀随机多项式，从而可复现地构造整个矩阵。
- $\text{ExpandS}(\rho_1)$ ：输入为短种子 ρ_1 ，输出为私钥秘密向量 $\mathbf{s}_1 \in \mathcal{S}^\ell$ 与 $\mathbf{s}_2 \in \mathcal{S}^k$ 。其实现方式是以 ρ_1 为根种子，为两个多项式向量的每个分量派生子种子，再分别采样系数落在 $\mathcal{S}$ 上的多项式向量。
- $\text{ExpandMask}(\rho', \kappa, a, b)$ ：输入为签名会话种子 ρ' 、重试计数器 κ 以及中心多项分布参数 (a, b) ，输出为服从 $\mathcal{D}_{a,b}$ 的掩码多项式向量。其实现方式是以 ρ' 为根种子，结合计数器 κ 为每个分量派生子种子，再根据子种子按中心多项分布采样。计数器 κ 在每次签名重试时递增，以保证不同重试使用的掩码不同。

2.4 挑战多项式的生成

挑战多项式 c 的生成方式沿用 ML-DSA 算法的采样方式，详见[算法 1](#)。

2.5 高低比特与提示

定义 2.1 (高低比特函数)。令 $r \in \mathbb{Z}, \alpha$ 是一个正偶数，则 r 的高低比特分别被定义为：

$$\text{HighBits}(r, \alpha) = r_1, \text{LowBits}(r, \alpha) = r_0,$$

其中， $r_1 = \lfloor \frac{r + \frac{\alpha}{2}}{\alpha} \rfloor, r_0 = r \bmod \pm \alpha$ 。

算法 1 挑战采样算法 $SampleInBall(B, \tau)$

```

1:  $c_0 c_1 \cdots c_{n-1} = 00 \cdots 0$ 
2: for  $i = n - \tau$  to  $n - 1$  do
3:    $j \leftarrow \{0, 1, \dots, i\}$  ▷ 通过随机字节  $B$  进行拒绝采样
4:    $s \leftarrow \{0, 1\}$ 
5:    $c_i = c_j$ 
6:    $c_j = (-1)^s$ 
7: end for
8: return  $c$ 

```

当 $\mathbf{r}$ 是一个多项式向量时, $HighBits(\mathbf{r}, \alpha)$ 表示对该多项式向量的每个多项式的每一个系数均使用 $HighBits$ 操作, 类似地可以推广到 $LowBits(\mathbf{r}, \alpha)$ 函数。对于高低比特函数, 有如下引理:

引理 2.1. 令正偶数 $\alpha > 2$ 以及素数 q 满足 $\alpha | 2(q-1)$, 则对于整数 $r \in \mathbb{Z}$, 其高低比特 $r_1 = HighBits(r, \alpha), r_0 = LowBits(r, \alpha)$ 满足如下性质:

1. $r_0 \in [-\frac{\alpha}{2}, \frac{\alpha}{2})$
2. $r = r_0 + \alpha \cdot r_1$
3. $r \in [0, 2q-1] \Rightarrow r_1 \in [0, \frac{2q-2}{\alpha}]$

证明. 根据第2章 $\bmod^\pm$ 的定义, 知性质 1 成立。由存在整数 p 使得 $r = r_0 + p\alpha$ 且 $r_0 \in [-\frac{\alpha}{2}, \frac{\alpha}{2})$ 知

$$r_1 = \lfloor \frac{r + \alpha/2}{\alpha} \rfloor = p + \lfloor \frac{r_0 + \alpha/2}{\alpha} \rfloor = p,$$

即性质 2, $r = r_0 + \alpha \cdot r_1$ 成立。由 $\alpha | 2(q-1)$ 知, 当 $r = 2q-1$ 时, 知 $r_1 = \lfloor \frac{2q-1+\alpha/2}{\alpha} \rfloor = \frac{2(q-1)}{\alpha} + \lfloor \frac{\alpha/2+1}{\alpha} \rfloor = \frac{2q-2}{\alpha}$; 当 $r = 0$ 时, $r_1 = \lfloor \frac{r+\alpha/2}{\alpha} \rfloor = 0$ 。结合 r_1 是一个关于 r 的非减函数知 $r_1 \in [0, \frac{2q-2}{\alpha}]$, 即性质 3 成立。□

在签名生成算法中, hint 向量 $\mathbf{h}$ 的计算只需要计算高比特, 为了进一步降低高比特值的熵, 对上述高低比特函数进行微调, 获得了 hint 高低比特函数, 详见定义 2.2。

定义 2.2 (hint 高低比特函数). 令正偶数 $\alpha > 2$ 以及素数 q 满足 $\alpha | 2(q-1)$, 令 $p = \frac{2(q-1)}{\alpha}, r \in \mathbb{Z}, r_1 = HighBits(r \bmod 2q, \alpha), r_0 = LowBits(r \bmod 2q, \alpha)$, 则 r 的 hint 高低比特分别被定义为:

$$HighBits^h(r, \alpha) = r'_1, LowBits^h(r, \alpha) = r'_0,$$

其中, 若 $r_1 = p$ 时, $r'_1 = 0, r'_0 = r_0 - 2$, 否则, $r'_1 = r_1, r'_0 = r_0$ 。

当 $\mathbf{r}$ 时一个多项式向量时, $HighBits^h(\mathbf{r}, \alpha)$ 表示对该多项式向量的每个多项式的每一个系数均使用 $HighBits^h$ 操作, 类似地可以推广到 $LowBits^h(\mathbf{r}, \alpha)$ 函数。对于 hint 高低比特函数, 有如下引理:

引理 2.2. 令正偶数 $\alpha > 2$ 以及素数 q 满足 $\alpha | 2(q-1)$, 令 $p = \frac{2(q-1)}{\alpha}$, 则对于整数 $r \in \mathbb{Z}$, 其 *hint* 高低比特 $r'_1 = \text{HighBits}^h(r \bmod 2q, \alpha)$, $r'_0 = \text{LowBits}^h(r \bmod 2q, \alpha)$ 满足如下性质:

1. $r'_0 \in [-\frac{\alpha}{2}, \frac{\alpha}{2})$
2. $r = \alpha \cdot r'_1 + r'_0 \bmod 2q$
3. $r'_1 \in [0, p-1]$

证明. 根据引理 2.1 知 $r_0 \in [-\frac{\alpha}{2}, \frac{\alpha}{2})$, 即性质 1 成立。根据引理 2.1 知 $r = \alpha \cdot r_1 + r_0 \bmod 2q$, 当 $r_1 \neq p$ 时, $r = \alpha \cdot r'_1 + r'_0 \bmod 2q$, 当 $r_1 = p$ 时, 则 $r = \alpha \cdot r_1 + r_0 = 2(q-1) + r'_0 + 2 = r'_0 \bmod 2q$, 即性质 2 成立。若 $r_1 = p$, 则 $r'_1 = 0 \in [0, p-1]$, 否则, $r'_1 = r_1 < p$, 即性质 3 成立。□

2.6 rANS 编码

CS 签名使用范围非对称数制系统 (rANS) [Dud09, Gie14] 对签名中的 $\mathbf{z}_0, \mathbf{z}_1$ 进行压缩, 使得签名尺寸更小。

定义 2.3 (范围非对称数制系统 (rANS) 压缩参数). 设 $t > 0$ 且 $S \subseteq [0, 2^t - 1]$. 设 $g: [0, 2^t - 1] \rightarrow \mathbb{Z} \cap (0, 2^t]$ 满足 $\sum_{x \in S} g(x) = 2^t$ 且对所有 $x \notin S$ 有 $g(x) = 0$ 。我们定义如下:

- $CDF: S \rightarrow \mathbb{Z}$, 定义为 $CDF(s) = \sum_{y=0}^{s-1} g(y)$ 。
- $D^{-1}: \mathbb{Z} \rightarrow S$, 其中 $D^{-1}(y)$ 定义为满足 $CDF(s) \leq y < CDF(s+1)$ 的 $s \in S$ 。
- $C: \mathbb{Z} \times S \rightarrow \mathbb{Z}$, 定义为

$$C(x, s) = \left\lfloor \frac{x}{g(s)} \right\rfloor \cdot 2^t + (x \bmod g(s)) + CDF(s). \quad (1)$$

称 $\text{params} = (t, S, g, CDF)$ 为 *rANS* 的压缩参数。*rANS* 编码与解码使用同一组预先约定的压缩参数, 编码将符号序列映射为一个整数; 解码则从该整数中恢复原始序列, 详见 算法 2 和 3。

算法 2 rANS 编码 $\text{rANS.Encode}(\mathbf{s})$

- 1: 解析 $\mathbf{s} = (s_1, \dots, s_m) \in S^m$
 - 2: $x_0 \leftarrow 0$
 - 3: **for** $i = 0$ **to** $m - 1$ **do**
 - 4: $x_{i+1} \leftarrow \lfloor x_i / g(s_{i+1}) \rfloor \cdot 2^t + (x_i \bmod g(s_{i+1})) + CDF(s_{i+1})$
 - 5: **end for**
 - 6: **return** x_m
-

2.7 Reject 算法

Reject 算法的输入为 $\mathbf{z}_{i+1}, c'_i \mathbf{s}, i$, 为了描述方便, $\mathbf{z}_{i+1} \triangleq \mathbf{x}, c'_i \mathbf{s} \triangleq \mathbf{r}$, 并将其看作向量来处理。设 $d_1 = \ell n, d_2 = kn, d = (1 + \ell + k)n$, 将 d 维向量 $\mathbf{x}$ 依次划分为三段: $\mathbf{x}_0 \in \mathbb{Z}^n, \mathbf{x}_1 \in \mathbb{Z}^{d_1}, \mathbf{x}_2 \in \mathbb{Z}^{d_2}$,

算法 3 rANS 解码 $\text{rANS.Decode}(x)$

```

1:  $y_0 \leftarrow x$ 
2:  $i \leftarrow 0$ 
3: while  $y_i > 0$  do
4:    $s'_{i+1} \leftarrow D^{-1}(y_i \bmod 2^t)$ 
5:    $y_{i+1} \leftarrow \lfloor y_i / 2^t \rfloor \cdot g(s'_{i+1}) + (y_i \bmod 2^t) - \text{CDF}(s'_{i+1})$ 
6:    $i \leftarrow i + 1$ 
7: end while
8:  $m \leftarrow i - 1$ 
9: return  $(s'_m, \dots, s'_1)$ 

```

$\mathbf{r}$ 也做相应的处理。令 $\tau_i = \min(i\tau', \tau)$, 定义一个分布 $\mathcal{D}'_i$ 是由 $\overline{\mathcal{D}}_{a_0, b_0}^{B_0-i} \times \overline{\mathcal{D}}_{a_1, b_1}^{B_1-\tau_i} \times \overline{\mathcal{D}}_{a_2, b_2}^{B_2-\tau_i}$ 以 $1/2$ 的概率平移 $\mathbf{r}$ 且以 $1/2$ 的概率平移 $-\mathbf{r}$ 获得。相应地定义另一个定义域更小的分布

$$\mathcal{D}_i = \overline{\mathcal{D}}_{a_0, b_0}^{B_0-(i+1)} \times \overline{\mathcal{D}}_{a_1, b_1}^{B_1-\tau_{i+1}} \times \overline{\mathcal{D}}_{a_2, b_2}^{B_2-\tau_{i+1}}.$$

并定义这两个分布的概率质量函数分别为 $g'_i(\mathbf{x}), g_i(\mathbf{x})$ 。令

$$A_0^i = \sum_{y=-B_0+i}^{B_0-i} f_{a_0, b_0}(y), A_1^i = \sum_{y=-B_1+\tau_i}^{B_1-\tau_i} f_{a_1, b_1}(y), A_2^i = \sum_{y=-B_2+\tau_i}^{B_2-\tau_i} f_{a_2, b_2}(y), A^i = (A_0^i)^n (A_1^i)^{d_1} (A_2^i)^{d_2}$$

$$g_0(\mathbf{x}_0) = g_{a_0, b_0}(\mathbf{x}_0), g_1(\mathbf{x}_1) = g_{a_1, b_1}(\mathbf{x}_1), g_2(\mathbf{x}_2) = g_{a_2, b_2}(\mathbf{x}_2), g(\mathbf{x}) = g_0(\mathbf{x}_0)g_1(\mathbf{x}_1)g_2(\mathbf{x}_2).$$

则根据附录A知 $\mathbf{x}_0 \sim \overline{\mathcal{D}}_{a_0, b_0}^{B_0-i}$ 的概率质量函数为 $\frac{g_0(\mathbf{x}_0)}{(A_0^i)^n}$, $\mathbf{x}_1 \sim \overline{\mathcal{D}}_{a_1, b_1}^{B_1-\tau_i}$ 的概率质量函数为 $\frac{g_1(\mathbf{x}_1)}{(A_1^i)^{d_1}}$, $\mathbf{x}_2 \sim \overline{\mathcal{D}}_{a_2, b_2}^{B_2-\tau_i}$ 的概率质量函数为 $\frac{g_2(\mathbf{x}_2)}{(A_2^i)^{d_2}}$. 记分布 $\mathcal{D}_i$ 的定义域为 C_i , 则对于任意的 $\mathbf{x} \sim \mathcal{D}_i$ 有

$$|(\mathbf{x}_0)_j| \leq B_0 - i - 1, |(\mathbf{x}_1)_j| \leq B_1 - \tau_{i+1}, |(\mathbf{x}_2)_j| \leq B_2 - \tau_{i+1},$$

且 $\mathbf{x} \sim \mathcal{D}_i$ 的概率

$$g_i(\mathbf{x}) = \frac{g_0(\mathbf{x}_0)g_1(\mathbf{x}_1)g_2(\mathbf{x}_2)}{(A_0^{i+1})^n (A_1^{i+1})^{d_1} (A_2^{i+1})^{d_2}} = \frac{g(\mathbf{x})}{A^{i+1}}.$$

记分布 $\mathcal{D}'_i$ 的定义域为 C'_i , 则 $C_i \subseteq C'_i$, 当 $\mathbf{x} \in C_i$ 时, $\mathbf{x} + \mathbf{r} \in C'_i, \mathbf{x} - \mathbf{r} \in C'_i$, 此时, $\mathbf{x} \sim \mathcal{D}'_i$ 的概率质量函数

$$g'_i(\mathbf{x}) = \frac{g(\mathbf{x} + \mathbf{r}) + g(\mathbf{x} - \mathbf{r})}{2A^i}.$$

Reject 算法的输入 $\mathbf{x} \sim \mathcal{D}'_i$, Reject 算法 (详见算法 4) 以接近 $p_i = \frac{g_i(\mathbf{x})}{M_i g'_i(\mathbf{x})}$ 的概率输出 False. 其中

$$p_i = \frac{g_i(\mathbf{x})}{M_i g'_i(\mathbf{x})} = \begin{cases} \frac{2A^i}{M_i A^{i+1}} \frac{g(\mathbf{x})}{g(\mathbf{x} + \mathbf{r}) + g(\mathbf{x} - \mathbf{r})}, & \mathbf{x} \in C_i \\ 0, & \mathbf{x} \in C'_i \setminus C_i \end{cases}$$

当 $\mathbf{x} \in C'_i \setminus C_i$ 时, $g(\mathbf{x}) = 0$, 即 $p_i = 0$, 即 Reject 算法总是返回 True, 算法 4 的第 1 步达到了该目标 (为了描述方便, 后面我们仅考虑 $\mathbf{x} \in C_i$ 的情况)。下面分析 Reject 算法如何在全流程无浮点运算的情况下近似地计算 p_i , 并证明该近似的相对误差小于 $2^{-L+4+\log d}$ 。

令 $M_i = \frac{2^L}{m_i}$ 使得 $p_i \leq 1$ 恒成立且 $m_i, L = 64$ 是整数, $\mathbf{x} \in C_i$ 时有

$$p_i = \frac{g_i(\mathbf{x})}{M_i g'_i(\mathbf{x})} = \frac{m_i A^i}{2^{L-1} A^{i+1}} \frac{g(\mathbf{x})}{g(\mathbf{x} + \mathbf{r}) + g(\mathbf{x} - \mathbf{r})} = \frac{m_i (A_0^i)^n (A_1^i)^{d_1} (A_2^i)^{d_2}}{2^{L-1} (A_0^{i+1})^n (A_1^{i+1})^{d_1} (A_2^{i+1})^{d_2}} \frac{g(\mathbf{x})}{g(\mathbf{x} + \mathbf{r}) + g(\mathbf{x} - \mathbf{r})}.$$

进一步地, 定义如下四个正整数列表:

$$\begin{aligned} list_0 &= [m_i] \cup [A_p^i]_{p,j} \cup [f_{a_p, b_p}((\mathbf{x}_p)_j)]_{p,j}, \\ list_1 &= [2^{L-1}] \cup [A_p^{i+1}]_{p,j}, \\ list_2 &= [f_{a_p, b_p}((\mathbf{x}_p + \mathbf{r}_p)_j)]_{p,j}, \\ list_3 &= [f_{a_p, b_p}((\mathbf{x}_p - \mathbf{r}_p)_j)]_{p,j}. \end{aligned}$$

根据附录A知, $g(\mathbf{x}) = \prod_{j=1}^n f_{a_0, b_0}((\mathbf{x}_0)_j) \prod_{j=1}^{d_1} f_{a_1, b_1}((\mathbf{x}_1)_j) \prod_{j=1}^{d_2} f_{a_2, b_2}((\mathbf{x}_2)_j)$, 因此,

$$p_i = \frac{\prod_{y \in list_0} y}{\prod_{y \in list_1} y (\prod_{y \in list_2} y + \prod_{y \in list_3} y)}. \quad (2)$$

Reject 算法 (详见算法 4) 的目标就是在无浮点操作下计算 p_i 的近似值 $\frac{N}{D}$, 并以 $\frac{N}{D}$ 的概率返回 False。公式2中, 每个 y 均为整数, 所以其分子分母均为整数, 可以直接通过生成随机比特加拒绝的方式在无浮点操作下以概率 $\frac{N}{D}$ 输出 False。然而上述方法的一个关键缺陷是分子分母的计算涉及多个整数的连乘运算, 准确计算该值会涉及到超大整数的乘法, 这会显著降低算法的效率。为此我们使用算法 5 来近似计算连乘操作以避免超大整数, 使得上述连乘对象只涉及 L 比特以内的整数, 从而提升计算效率。具体来说, 给定一个正整数列表 $list$, 算法 5 的主要思路先将每个列表元素截断为 L 比特以内 (这步通过预计算完成), 然后使用迭代的方式计算 $list$ 中前 i 个元素的乘积, 若该乘积超过 L 比特, 则将其截断为 L 比特, 然后计算该值与 $list[i+1]$ 的乘积, 最终算法 5 输出一个不长于 L 比特长的大整数 N' 以及一个整数 coe , 并能使用 $2^{coe} N'$ 来近似计算 $\prod_{x \in list} x$ 。算法 4 的第 1-12 行通过多次调用算法 5 获得了 p_i 的近似值 $\frac{N}{D}$, 并通过 13-21 步的随机比特采样及拒绝使得算法 4 以 $\frac{N}{D}$ 的概率输出 False。根据引理 B.2 知, $\frac{N}{D}$ 相对于 p_i 的相对误差小于 $2^{-L+4+\log d}$ 。

3 签名

基于第2章的基础知识与辅助函数, 本章描述 CS 签名算法。其中3.1小节描述了 CS 的核心算法——拒绝采样算法, 3.2节描述简化版 CS 签名算法以突出其设计原理与创新点, 3.3节在简化版 CS 签名算法的基础上加入了随机数的生成与使用、公钥压缩以及签名压缩等, 描述了完整的 CS 签名算法。3.4节证明了 CS 签名算法的正确性, 并给出了 CS 签名算法的签名重复次数。

3.1 拒绝采样算法

FS 类格签名算法的一个关键算法是拒绝采样算法, CS 算法也同样如此, CS 算法的目标是实现全流程无浮点运算的双峰 FS 类格签名算法, 这使得它与传统双峰 FS 拒绝采样算法明显不同。形式上看, 我们的拒绝采样算法 (算法 6) 是传统双峰拒绝采样算法 [DDLL13] 与迭代拒绝采样算法 [Gar25] 的结合。具体来说, CS 拒绝采样算法是将迭代拒绝采样算法 [Gar25] 中取到

算法 4 *Reject*($\mathbf{x}, \mathbf{r}, j$)

```

1: if  $\|\mathbf{z}_{i+1}^0\|_\infty > B_0 - (i+1)$  or  $\|\mathbf{z}_{i+1}^1\|_\infty > B_1 - \tau_{i+1}$  or  $\|\mathbf{z}_{i+1}^2\|_\infty > B_2 - \tau_{i+1}$  then
2:   return True  $\triangleright \mathbf{z}_{i+1} \in C'_i \setminus C_i$ 
3: end if
4:  $N, coe_0 = Approx(list_0)$ 
5:  $N_1, coe_1 = Approx(list_1)$ 
6:  $N_2, coe_2 = Approx(list_2)$ 
7:  $N_3, coe_3 = Approx(list_3)$ 
8:  $coe_4 = \min(coe_2, coe_3)$ 
9:  $D, coe_5 = Approx([N_1, 2^{coe_2 - coe_4} N_2 + 2^{coe_3 - coe_4} N_3])$ 
10:  $coe = coe_0 - coe_1 - coe_4 - coe_5$ 
11: if  $coe \geq 0$  then
12:    $N = 2^{coe} N$ 
13: else
14:    $D = 2^{-coe} D$ 
15: end if
16:  $bits = \text{len}(D)$   $\triangleright \text{len}(x)$  表示  $x$  的比特数
17:  $r \leftarrow [2^{bits}]$ 
18: if  $r \geq D$  then
19:   Goto step 14
20: end if
21: if  $r \geq N$  then
22:   return True
23: else
24:   return False
25: end if

```

算法 5 *Approx(list)*

```

1:  $d' = \text{len}(\text{list})$ 
2:  $N' = 1$ 
3:  $\text{coe} = 0$ 
4: for  $i$  from 0 to  $d' - 1$  do
5:    $s = \text{len}(\text{list}[i]) - L$   $\triangleright \text{len}(x)$  表示  $x$  的比特数
6:   if  $s > 0$  then
7:      $\text{list}[i] \gg s$ 
8:      $\text{coe} += s$ 
9:   end if
10:   $N' = N' \cdot \text{list}[i]$ 
11:   $s = \text{len}(N') - L$ 
12:  if  $s > 0$  then
13:     $N' \gg s$   $\triangleright N'$  右移  $s$  比特
14:     $\text{coe} += s$ 
15:  end if
16: end for
17: return  $N', \text{coe}$ 

```

$\mathbf{z} = \mathbf{y} + \mathbf{c}\mathbf{s}$ 与 $\mathbf{z} = \mathbf{y} - \mathbf{c}\mathbf{s}$ 的概率像 [DDLL13] 一样均设为 $\frac{1}{2}$ 。此外，通过引入一个参数 τ' 使得迭代拒绝更灵活，同时通过调节该参数获得更高效的 CS 算法参数。

令 c 中非零元素的位置为 $j_0, \dots, j_{\tau-1}$ (τ 等算法参数详见表3), c'_i 在 $j_{i\tau'}, \dots, j_{\min((i+1)\tau'-1, \tau-1)}$ 处与 c 相同 (c'_i 的汉明重量不超过 τ')，其余位置均为 0，因此， $c = c'_0 + \dots + c'_{N-1}$ 。输入 $\mathbf{s}, \mathbf{y}, c, \tau'$ ，算法 6 先通过上述方式获得 c'_i ，并进行 $N = \lceil \frac{\tau}{\tau'} \rceil$ 次迭代。在每次迭代中，首先计算 $\mathbf{z}_{i+1} = \mathbf{z}_i \pm c'_i \mathbf{s}$ (第 5 步)，并在第 9 步确定是否对其进行拒绝。由于 $\mathbf{y} = (\mathbf{y}_0, \mathbf{y}_1^\top, \mathbf{y}_2^\top)^\top$ 的三个分量分别取自于分布 $\mathcal{D}_{a_0, b_0}^{\mathcal{R}}, \mathcal{D}_{a_1, b_1}^{\mathcal{R}^\ell}, \mathcal{D}_{a_2, b_2}^{\mathcal{R}^k}$ ，结合 2.7 节的定义知，第 8 步后的 $\mathbf{z}_{i+1} \sim \mathcal{D}'_i$ ，该分布由 $\overline{\mathcal{D}}_{a_0, b_0}^{B_0-i} \times \overline{\mathcal{D}}_{a_1, b_1}^{B_1-\tau_i} \times \overline{\mathcal{D}}_{a_2, b_2}^{B_2-\tau_i}$ 以 $1/2$ 的概率平移 $c'_i \mathbf{s}$ 且以 $1/2$ 的概率平移 $-c'_i \mathbf{s}$ 获得，其中 $\tau_i = \min(i\tau', \tau)$ 。进一步根据 2.7 小节的定义知，Reject 算法的输入 $\mathbf{z}_{i+1}$ 满足

$$\mathbf{z}_{i+1} \sim \mathcal{D}'_i,$$

其概率质量函数为 $g'_i(\mathbf{z}_{i+1})$ 。沿用 2.7 小节的定义，令

$$\mathcal{D}_i = \overline{\mathcal{D}}_{a_0, b_0}^{B_0-(i+1)} \times \overline{\mathcal{D}}_{a_1, b_1}^{B_1-\tau_{i+1}} \times \overline{\mathcal{D}}_{a_2, b_2}^{B_2-\tau_{i+1}}$$

且 $\mathbf{z}_{i+1} \sim \mathcal{D}_i$ 的概率质量函数为 $g_i(\mathbf{z}_{i+1})$ 。紧接着，若第 9 步的 Reject 算法（详见算法 4）以概率

$$p_i = \frac{g_i(\mathbf{z}_{i+1})}{M_i g'_i(\mathbf{z}_{i+1})} \quad (3)$$

输出 False (M_i 是常数使得 $p_i \leq 1$ 总成立)，根据引理 B.4 知，此时，非 $\perp$ 的 $\mathbf{z}_{i+1}$ 服从分布 $\mathcal{D}_i$ 且第 i 次迭代中第 9 步的重复次数为 M_i 。因此，Reject 算法将上述 $\mathbf{z}_{i+1}$ 拒绝为与私钥 $\mathbf{s}, c'_i, \text{bit}_i$ 均独立的截断中心多项分布 $\mathcal{D}_i$ 。然而， p_i 的精确计算涉及到多个整数的连乘操作，为了避开超大

整数操作, 算法 4 输出 False 的概率为 $\frac{N}{D}$ 。根据引理 B.2 知, p_i 与 $\frac{N}{D}$ 的相对误差小于 $2^{-L+4+\log d}$ ($d = n(k + \ell + 1)$), 进一步, 根据引理 B.3 知, 该近似至多降低 CS 算法 1 比特的实际安全强度。

算法 6 拒绝采样算法 $RejectSample(\mathbf{s}, \mathbf{y}, c, \tau')$

```

1:  $N = \lceil \frac{\tau'}{\tau} \rceil$ 
2: if  $\|\mathbf{y}_0\|_\infty > B_0$  or  $\|\mathbf{y}_1\|_\infty > B_1$  or  $\|\mathbf{y}_2\|_\infty > B_2$  then
3:   return  $\perp$ 
4: end if
5:  $\mathbf{z}_0 = \mathbf{y}$ 
6: for  $i = 0$  to  $N - 1$  do
7:    $bit_i \leftarrow \{0, 1\}$ 
8:    $\mathbf{z}_{i+1} = \mathbf{z}_i + (-1)^{bit_i} c'_i \mathbf{s}$   $\triangleright \mathbf{z}_{i+1}$  由  $\mathbf{z}_i$  以  $\frac{1}{2}$  的概率平移  $c'_i \mathbf{s}$  或以  $\frac{1}{2}$  的概率平移  $-c'_i \mathbf{s}$  获得
9:   if  $Reject(\mathbf{z}_{i+1}, c'_i \mathbf{s}, i) == True$  then  $\triangleright$  输出 False 的概率为  $\frac{N}{D} \approx p_i = \frac{g_i(\mathbf{z}_{i+1})}{M_i g'_i(\mathbf{z}_{i+1})}$ 
10:    return  $\perp$   $\triangleright \mathbf{z}_{i+1} \sim \mathcal{D}_i$ 
11:  end if
12: end for
13: return  $\mathbf{z}_N, c_r = \sum_{i=0}^{N-1} (-1)^{bit_i} c'_i$ 

```

3.2 简化 CS 签名算法

简化密钥生成算法 (详见算法 7) 沿用 HAETA 算法 [CHE⁺24] 的密钥生成思想, 使用 MLWE 困难问题, 即 CS 签名算法的公钥为 $(\mathbf{A}', \mathbf{b})$, 其中 $\mathbf{b} = \mathbf{A}' \mathbf{s}_1 + \mathbf{s}_2$ 。该算法第 5 步多项式向量 $\mathbf{j} \in \mathcal{R}^k$ 的第一个多项式分量为 1, 其他多项式分量均为 0。该密钥生成算法输出的 $\mathbf{A}, \mathbf{s}$ 满足

$$\mathbf{A} \mathbf{s} = q \mathbf{j} - 2 \mathbf{b} + 2 \mathbf{A}' \mathbf{s}_1 + 2 \mathbf{s}_2 = q \mathbf{j} \bmod 2q.$$

算法 7 简化版密钥生成算法 $KeyGenSimple$

```

1:  $\mathbf{A}' \leftarrow \mathcal{R}_q^{k \times \ell}$ 
2:  $\mathbf{s}_1 \leftarrow \mathcal{S}^\ell$ 
3:  $\mathbf{s}_2 \leftarrow \mathcal{S}^k$ 
4:  $\mathbf{b} = \mathbf{A}' \mathbf{s}_1 + \mathbf{s}_2 \in \mathcal{R}_q^k$ 
5:  $\mathbf{A} = (q \mathbf{j} - 2 \mathbf{b} | 2 \mathbf{A}' | 2 \mathbf{I}_k) \in \mathcal{R}_{2q}^{k \times (1 + \ell + k)}$ 
6:  $\mathbf{s} = (1, \mathbf{s}_1^\top, \mathbf{s}_2^\top)^\top \in \mathcal{R}^{k + \ell + 1}$ 
7: return  $pk = (\mathbf{A}', \mathbf{b}), sk = \mathbf{s}$ 

```

我们目标是设计一个全流程无浮点运算的紧致双峰 FS 类格签名算法, 设计的核心在于签名生成算法的不同。双峰 FS 格签名算法的签名生成过程常涉及浮点运算, 包括掩码向量 $\mathbf{y}$ 的生成以及计算签名 $\mathbf{z}$ 的拒绝概率, 为了避免这两步涉及浮点运算, CS 签名算法进行了如下设计:

1. 掩码向量 $\mathbf{y}$ 使用一个简单易整数采样的中心多项分布;

2. 基于 Rényi 散度，提出了一种仅通过整数运算来计算签名拒绝概率近似值的方法。

通过上述设计，使得 CS 签名算无浮点运算。此外，我们将双峰 FS 签名算法挑战多项式 c 的系数空间，从 $\{0, 1\}$ 扩展到 $\{0, \pm 1\}$ ，降低了挑战多项式的汉明重量，从而使得 CS 签名算法更紧致。为了提升签名算法的效率，CS 签名算法沿用迭代双峰 [Gar25] 的思想，并通过引入一个新参数 τ' ，提升了迭代双峰的灵活性。

简化版签名生成算法（详见算法 8）的前 3 步主要用于生成服从中心多项分布的掩码向量 $\mathbf{y}$ 。算法 8 的主要难点在于第 6 步拒绝采样算法 *RejectSample* 生成签名 $\mathbf{z} = (\mathbf{z}_0, \mathbf{z}_1^\top, \mathbf{z}_2^\top)^\top \in \mathcal{R} \times \mathcal{R}^\ell \times \mathcal{R}^k$ 。为了高效且安全地实现拒绝采样算法，我们在全流程无浮点运算情况下完成了对签名拒绝概率的近似计算，并通过控制相对误差在 Rényi 散度下证明了该近似运算最多降低算法 1 比特的实际安全强度（详见即算法 6）。

算法 8 简化版签名生成算法 *SignSimple*($\mathbf{A}, \mathbf{s}, M$)

```

1:  $\mathbf{y}_0 \leftarrow \mathcal{D}_{a_0, b_0}^{\mathcal{R}}$ 
2:  $\mathbf{y}_1 \leftarrow \mathcal{D}_{a_1, b_1}^{\mathcal{R}^\ell}$ 
3:  $\mathbf{y}_2 \leftarrow \mathcal{D}_{a_2, b_2}^{\mathcal{R}^k}$ 
4:  $\mathbf{y} = (\mathbf{y}_0, \mathbf{y}_1^\top, \mathbf{y}_2^\top)^\top \in \mathcal{R}^{1+\ell+k}$ 
5:  $\mathbf{w} = \mathbf{A}\mathbf{y} \bmod 2q$ 
6:  $\tilde{c} = H(M, \mathbf{w}) \in \{0, 1\}^{2\lambda}$ 
7:  $c = \text{SampleInBall}(\tilde{c}, \tau)$ 
8:  $\mathbf{z}, c_r = \text{RejectSample}(\mathbf{s}, \mathbf{y}, c, \tau')$ 
9: if  $\mathbf{z} = \perp$  then
10:   Goto step 1
11: end if
12: return  $(\mathbf{z}, \tilde{c})$  ▷ rANS 编码

```

签名验证算法比较简单（详见算法 9），和传统双峰 FS 格签名算法类似，区别主要在于 CS 签名验证算法对签名 $\mathbf{z}$ 的无穷范数进行界定，而非 2 范数。

算法 9 简化版签名验证算法 *VerifySimple*($\mathbf{A}, M, \mathbf{z}, \tilde{c}$)

```

1:  $c' = \text{SampleInBall}(\tilde{c}, \tau)$ 
2:  $\mathbf{w}' = \mathbf{A}\mathbf{z} - qc'\mathbf{j} \bmod 2q$ 
3: if  $\|\mathbf{z}_0\|_\infty > B_0 - \lceil \frac{\tau}{\tau'} \rceil$  or  $\|\mathbf{z}_1\|_\infty > B_1 - \tau$  or  $\|\mathbf{z}_2\|_\infty > B_2 - \tau$  or  $\tilde{c} \neq H(M, \mathbf{w}')$  then
4:   Reject
5: else
6:   Accept
7: end if

```

3.3 完整 CS 签名算法

密钥生成算法 10 在简化版密钥生成算法 7 的基础上, 对公钥进行了压缩, 即将公钥中的 $\mathbf{b}$ 变为 $\mathbf{b}_1$, 其中, $\mathbf{b}_1 = \text{HighBits}(\mathbf{b}, \beta)$, $\mathbf{b}_0 = \text{LowBits}(\mathbf{b}, \beta)$, β 是 2 的幂次, 此时, 有

$$\mathbf{A}\mathbf{s} = q\mathbf{j} + 2\mathbf{b}_0 \bmod 2q.$$

此外, 还加入了随机数的生成以及随机元素 $\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2$ 的生成过程, 并将公钥 pk 中的 $\mathbf{A}'$ 使用种子 ρ_0 来替代, 还在私钥 sk 中加入了 $K, tr, \rho_0, \mathbf{b}_0$ 。

算法 10 密钥生成算法 $\text{KeyGen}(1^\lambda)$

- 1: $\rho \leftarrow \{0, 1\}^{2\lambda}$
 - 2: $(\rho_0, \rho_1, K) := H(\rho, k, l) \in \{0, 1\}^\lambda \times \{0, 1\}^{2\lambda} \times \{0, 1\}^\lambda$
 - 3: $\mathbf{A}' := \text{ExpandA}(\rho_0) \in \mathcal{R}_q^{k \times \ell}$
 - 4: $(\mathbf{s}_1^\top, \mathbf{s}_2^\top)^\top := \text{ExpandS}(\rho_1) \in \mathcal{S}^\ell \times \mathcal{S}^k$
 - 5: $\mathbf{b} := \mathbf{A}'\mathbf{s}_1 + \mathbf{s}_2 \in \mathcal{R}_q^k$
 - 6: $\mathbf{A} = (q\mathbf{j} - 2\beta\mathbf{b}_1 | 2\mathbf{A}' | 2\mathbf{I}_k) \in \mathcal{R}_{2q}^{k \times (k+\ell+1)}$
 - 7: $\mathbf{s} = (1, \mathbf{s}_1^\top, \mathbf{s}_2^\top)^\top \in \mathcal{R}^{k+\ell+1}$
 - 8: $pk = (\rho_0, \mathbf{b}_1)$
 - 9: $tr := H(pk) \in \{0, 1\}^{2\lambda}$
 - 10: $sk = (\mathbf{s}_1, \mathbf{s}_2, K, tr, \rho_0, \mathbf{b}_0)$
 - 11: **return** pk, sk
-

签名生成算法 11 在简化版签名生成算法 8 的基础上, 使用压缩技术将签名中的环向量 $\mathbf{z}_2$ 替换为 $\mathbf{h}$, 并使用 RANS 编码技术对环元素 $\mathbf{z}_0$ 和环向量 $\mathbf{z}_1$ 进行了编码, 从而降低了签名的尺寸。此外, 还加入了矩阵 $\mathbf{A}$ 的生成以及掩码向量 $\mathbf{y}$ 的生成过程。由于密钥生成过程对公钥进行了压缩, 第 18 步中向量 $\mathbf{h}$ 的计算过程涉及到了私钥中的 $\mathbf{b}_0$ 。

基于签名生成算法 11, 签名验证算法 12 也在简化版签名验证算法 9 的基础上做了相应的调整。在算法 12 第 13 行中 $B'_2 = B_2 - \tau + \frac{\alpha}{4} + 1 + \tau \frac{\beta}{2}$ 。

3.4 正确性与签名重复次数

这一小节分析完整版 CS 签名算法的正确性和签名重复次数。

定理 3.1. 对于任意的公私钥对 $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ 、任意消息 M 和签名生成算法 $\text{Sign}(sk, M)$ 的输出 σ , 签名验证算法 $\text{Verify}(pk, M, \sigma)$ 总能验证通过。

证明. 首先证明签名验证算法第 13 步中 $\tilde{c} = H(\mu', \mathbf{w}', w')$ 总是成立。对于签名生成算法 $\text{Sign}(sk, M)$ 的输出 $\sigma = (\sigma_0, \sigma_1, \mathbf{h}, \tilde{c})$, 签名生成算法和签名验证算法中的符号满足如下条件:

$$\mu' = \mu, \mathbf{z}'_0 = \mathbf{z}_0, \mathbf{z}'_1 = \mathbf{z}_1, c' = \text{SampleInBall}(\tilde{c}, \tau) = c.$$

算法 11 签名生成算法 $Sign(sk, M)$

```

1:  $\mathbf{A}' := ExpandA(\rho_0) \in \mathcal{R}_q^{k \times \ell}$ 
2:  $\mathbf{A} = (q\mathbf{j} - 2\beta\mathbf{b}_1 | 2\mathbf{A}' | 2\mathbf{I}_k) \in \mathcal{R}_{2q}^{k \times (1+\ell+k)}$ 
3:  $\mu := H(tr, M) \in \{0, 1\}^{2\lambda}$ 
4:  $\kappa := 0$ 
5:  $\rho' := H(K, \mu) \in \{0, 1\}^{2\lambda}$  (随机签名中  $\rho' \leftarrow \{0, 1\}^{2\lambda}$ )
6:  $\mathbf{y}_0 := ExpandMask(\rho', \kappa, a_0, b_0) \sim \mathcal{D}_{a_0, b_0}^{\mathcal{R}}$ 
7:  $\mathbf{y}_1 := ExpandMask(\rho', \kappa, a_1, b_1) \sim \mathcal{D}_{a_1, b_1}^{\mathcal{R}^\ell}$ 
8:  $\mathbf{y}_2 := ExpandMask(\rho', \kappa, a_2, b_2) \sim \mathcal{D}_{a_2, b_2}^{\mathcal{R}^k}$ 
9:  $\mathbf{y} = (\mathbf{y}_0, \mathbf{y}_1^\top, \mathbf{y}_2^\top)^\top \in \mathcal{R}^{k+\ell+1}$ 
10:  $\mathbf{w} := \mathbf{A}\mathbf{y} \bmod 2q$ 
11:  $\tilde{c} := H(\mu, HighBits^h(\mathbf{w}, \alpha), LSB(\mathbf{y}_0)) \in \{0, 1\}^{2\lambda}$ 
12:  $c = SampleInBall(\tilde{c}, \tau)$ 
13:  $\mathbf{z}, c_r := RejectSample(\mathbf{s}, \mathbf{y}, c, \tau')$ 
14: if  $\mathbf{z} = \perp$  then
15:    $\kappa \leftarrow \kappa + 1$ 
16:   Goto step 6
17: end if
18:  $\mathbf{h} := HighBits^h(\mathbf{w}, \alpha) - HighBits^h(\mathbf{w} - 2\mathbf{z}_2 + 2c_r\mathbf{b}_0, \alpha) \bmod \pm \frac{2(q-1)}{\alpha}$ 
19:  $\sigma_0 := \text{rANS.Encode}(\mathbf{z}_0)$   $\triangleright \mathbf{z}_0 \in \mathcal{R}$ 
20:  $\sigma_1 := \text{rANS.Encode}(\mathbf{z}_1)$   $\triangleright \mathbf{z}_1 \in \mathcal{R}^\ell$ 
21: return  $\sigma = (\sigma_0, \sigma_1, \mathbf{h}, \tilde{c})$ 

```

算法 12 签名验证算法 $Verify(pk, M, \sigma)$

```

1:  $\mathbf{A}' := ExpandA(\rho_0)$ 
2:  $\mathbf{A}_0 = q\mathbf{j} - 2\beta\mathbf{b}_1 \in \mathcal{R}_{2q}^k$ 
3:  $\mathbf{A}_1 = 2\mathbf{A}' \in \mathcal{R}_{2q}^{k \times l}$ 
4:  $\mu' := H(H(pk), M)$ 
5:  $\mathbf{z}'_0 := \text{rANS.Decode}(\sigma_0)$ 
6:  $\mathbf{z}'_1 := \text{rANS.Decode}(\sigma_1)$ 
7:  $c' := \text{SampleInBall}(\tilde{c}, \tau)$ 
8:  $\mathbf{v} := \mathbf{A}_0\mathbf{z}'_0 + \mathbf{A}_1\mathbf{z}'_1 - qc'\mathbf{j} \bmod 2q$ 
9:  $\mathbf{w}' := \text{HighBits}^h(\mathbf{v}, \alpha) + \mathbf{h} \bmod \frac{2(q-1)}{\alpha}$ 
10:  $w' := \text{LSB}(\mathbf{z}'_0 - c')$ 
11:  $\mathbf{z}'_2 := (\alpha \cdot \mathbf{w}' + w'\mathbf{j} - \mathbf{v} \bmod 2q)/2 \bmod \pm q$ 
12: if  $\|\mathbf{z}'_0\|_\infty > B_0 - \lceil \frac{\tau}{\tau'} \rceil$  or  $\|\mathbf{z}'_1\|_\infty > B_1 - \tau$  or  $\|\mathbf{z}'_2\|_\infty > B'_2$  or  $\tilde{c} \neq H(\mu', \mathbf{w}', w')$  then
13:   Reject
14: else
15:   Accept
16: end if

```

由 $N = \lceil \frac{\tau}{\tau'} \rceil, c_r = \sum_{i=0}^{N-1} (-1)^{bit_i} c'_i$, 则签名验证算法中 $\mathbf{v}$ 满足:

$$\begin{aligned}
\mathbf{v} &= \mathbf{A}_0\mathbf{z}_0 + \mathbf{A}_1\mathbf{z}_1 - qc\mathbf{j} \bmod 2q \\
&= \mathbf{A}\mathbf{z} - \mathbf{A}_2\mathbf{z}_2 - qc\mathbf{j} \bmod 2q \\
&= \mathbf{A}(\mathbf{y} + c_r\mathbf{s}) - 2\mathbf{z}_2 - qc\mathbf{j} \bmod 2q \\
&= \mathbf{w} + c_r(q\mathbf{j} + 2\mathbf{b}_0) - 2\mathbf{z}_2 - qc\mathbf{j} \bmod 2q \\
&= \mathbf{w} - 2\mathbf{z}_2 + 2c_r\mathbf{b}_0 \bmod 2q.
\end{aligned}$$

进一步知

$$\begin{aligned}
\mathbf{w}' &= \text{HighBits}^h(\mathbf{v}, \alpha) + \mathbf{h} \bmod \frac{2(q-1)}{\alpha} \\
&= \text{HighBits}^h(\mathbf{w} - 2\mathbf{z}_2 + 2c_r\mathbf{b}_0, \alpha) + \mathbf{h} \bmod \frac{2(q-1)}{\alpha} \\
&= \text{HighBits}^h(\mathbf{w}, \alpha).
\end{aligned}$$

此外, 考虑到 $w' = \text{LSB}(\mathbf{z}_0 - c) = \text{LSB}(\mathbf{y}_0 + c_r - c) = \text{LSB}(\mathbf{y}_0)$ 知 $H(\mu', \mathbf{w}', w') = \tilde{c}$ 。

接下来证明签名验证算法第 13 步的边界判断条件总是成立, 根据[算法 6](#)的迭代拒绝过程知 $\|\mathbf{z}'_0\|_\infty \leq B_0 - N, \|\mathbf{z}'_1\|_\infty \leq B_1 - \tau, \|\mathbf{z}_2\|_\infty \leq B_2 - \tau$ 成立。根据

$$\text{LSB}(\mathbf{w}) = \text{LSB}((q\mathbf{j} - 2\beta\mathbf{b}_1)\mathbf{y}_0 + 2\mathbf{A}'\mathbf{y}_1 + 2\mathbf{y}_2) = \text{LSB}(\mathbf{y}_0)\mathbf{j},$$

结合第2章 hint 高低比特函数的性质知

$$\begin{aligned} \mathbf{z}'_2 &= (\alpha \cdot \mathbf{w}' + w' \mathbf{j} - \mathbf{v} \bmod 2q) / 2 \bmod^{\pm} q \\ &= (\alpha \cdot HighBits^h(\mathbf{w}, \alpha) + LSB(\mathbf{y}_0) \mathbf{j} - (\mathbf{w} - 2\mathbf{z}_2 + 2c_r \mathbf{b}_0) \bmod 2q) / 2 \bmod^{\pm} q \\ &= \mathbf{z}_2 - c_r \mathbf{b}_0 + (-LowBits^h(\mathbf{w}, \alpha) + LSB(\mathbf{w})) / 2 \bmod^{\pm} q. \end{aligned}$$

最后一个等式成立是因为在我们的参数设定中 $\|2\mathbf{z}_2 - 2c_r \mathbf{b}_0 - LowBits^h(\mathbf{w}, \alpha) + LSB(\mathbf{w})\|_{\infty} \leq 2q$. 结合 $LowBits^h(\mathbf{w}, \alpha) \in [-\frac{\alpha}{2} - 2, \frac{\alpha}{2}]$ 知 $\|\mathbf{z}'_2 - \mathbf{z}_2\|_{\infty} \leq \frac{\alpha}{4} + 1 + \tau \frac{\beta}{2}$, 进一步知 $\|\mathbf{z}'_2\|_{\infty} \leq \|\mathbf{z}_2\|_{\infty} + \|\mathbf{z}'_2 - \mathbf{z}_2\|_{\infty} \leq B'_2$. 从而完成了 CS 签名算法正确性的证明。□

CS 签名算法的签名重复次数主要源自于边界拒绝 (算法6第 2 步) 和概率拒绝 (算法6第 9 步) 两部分。边界拒绝前后的分布分别为

$$\mathcal{D}_{a_0, b_0} \times \mathcal{D}_{a_1, b_1} \times \mathcal{D}_{a_2, b_2}, \overline{\mathcal{D}}_{a_0, b_0}^{B_0} \times \overline{\mathcal{D}}_{a_1, b_1}^{B_1} \times \overline{\mathcal{D}}_{a_2, b_2}^{B_2}.$$

基于2.7的定义, 边界拒绝概率

$$p' = \frac{A^0}{2^{2(na_0b_0+d_1a_1b_1+d_2a_2b_2)}} \triangleq \frac{A^0}{A}.$$

沿用2.7小节的定义, 并令 $f_{a_0, b_0} = f_0, f_{a_1, b_1} = f_1, f_{a_2, b_2} = f_2$, 3.2给出了算法6第 i 次迭代中第 9 步的重复次数, 并给出了 CS 算法的签名重复次数。

定理 3.2. 对于任意的 $\mathbf{s}$ 以及 c , 在 CS 算法参数下, 算法6在第 i 次迭代时第 9 步的重复次数

$$M_i = \frac{A^i g(\mathbf{x}^i)}{A^{i+1} \sqrt{g_i(\mathbf{x}^i + \mathbf{r}^i) g(\mathbf{x}^i - \mathbf{r}^i)}},$$

其中 $\mathbf{r}^i = (\mathbf{r}_0^i, \mathbf{r}_1^i, \mathbf{r}_2^i)$, $(\mathbf{r}_0^i)_j = 1, (\mathbf{r}_1^i)_j = (\mathbf{r}_2^i)_j = \tau_i'', \tau_i'' = \min(\tau', \tau - i\tau')$, $\mathbf{x}^i = (\mathbf{x}_0^i, \mathbf{x}_1^i, \mathbf{x}_2^i)$, $(\mathbf{x}_p^i)_j$ 是函数 $\frac{f_p^2(x)}{f_p(x + (\mathbf{r}_p^i)_j) f_p(x - (\mathbf{r}_p^i)_j)}$ 的最大值点. 进一步地, CS 的重复次数 M 为

$$\frac{A}{A^N} \prod_{i=0}^{N-1} \frac{g(\mathbf{x}^i)}{\sqrt{g(\mathbf{x}^i + \mathbf{r}^i) g(\mathbf{x}^i - \mathbf{r}^i)}}.$$

证明. 根据2.7的定义知, $p_i = \frac{2A^i}{M_i A^{i+1}} \frac{g(\mathbf{x})}{g(\mathbf{x} + \mathbf{r}) + g(\mathbf{x} - \mathbf{r})}$, 进一步地根据引理 B.4知, 第 i 次迭代第 9 步的重复次数为 M_i , 其中 M_i 使得 $p_i \leq 1$. 因此,

$$M_i \geq \frac{2A^i}{A^{i+1}} \frac{g(\mathbf{x})}{g(\mathbf{x} + \mathbf{r}) + g(\mathbf{x} - \mathbf{r})} \geq \frac{A^i}{A^{i+1}} \frac{g(\mathbf{x})}{\sqrt{g(\mathbf{x} + \mathbf{r}) g(\mathbf{x} - \mathbf{r})}},$$

令

$$M'_i(\mathbf{x}, \mathbf{r}) = \frac{g(\mathbf{x})}{\sqrt{g(\mathbf{x} + \mathbf{r}) g(\mathbf{x} - \mathbf{r})}},$$

根据引理A.1知 M'_i 是关于 $|(\mathbf{r}_p)_j|$ 的增函数, 所以, 在 $\mathbf{r} = \mathbf{r}^i$ 处, $M'_i(\mathbf{x}, \mathbf{r})$ 取得最大值 $M'_i(\mathbf{x}, \mathbf{r}^i)$, 将其简记为 $M'_i(\mathbf{x})$. 则 $M'_i(\mathbf{x})$ 可被分解为 d 个因子的乘积

$$\prod_{j=1}^n \frac{f_0^2((\mathbf{x}_0)_j)}{f_0((\mathbf{x}_0)_j + (\mathbf{r}_0^i)_j) f_0((\mathbf{x}_0)_j - (\mathbf{r}_0^i)_j)} \prod_{j=1}^{d_1} \frac{f_1^2((\mathbf{x}_1)_j)}{f_1((\mathbf{x}_1)_j + (\mathbf{r}_1^i)_j) f_1((\mathbf{x}_1)_j - (\mathbf{r}_1^i)_j)} \prod_{j=1}^{d_2} \frac{f_2^2((\mathbf{x}_2)_j)}{f_2((\mathbf{x}_2)_j + (\mathbf{r}_2^i)_j) f_2((\mathbf{x}_2)_j - (\mathbf{r}_2^i)_j)}$$

因此，当每个函数 $\frac{f_p^2(x)}{f_p(x+(\mathbf{r}_p^i)_j)f_p(x-(\mathbf{r}_p^i)_j)}$ 获得最大值点时，参数 M_i 获得最小值，此时

$$M_i = \frac{A^i g(\mathbf{x}^i)}{A^{i+1} \sqrt{g_i(\mathbf{x}^i + \mathbf{r}^i)g(\mathbf{x}^i - \mathbf{r}^i)}}.$$

进一步知，总签名重复次数

$$M = \frac{1}{p'} \prod_{i=0}^{N-1} \frac{A^i g(\mathbf{x}^i)}{A^{i+1} \sqrt{g_i(\mathbf{x}^i + \mathbf{r}^i)g(\mathbf{x}^i - \mathbf{r}^i)}} = \frac{A}{A^N} \prod_{i=0}^{N-1} \frac{g(\mathbf{x}^i)}{\sqrt{g(\mathbf{x}^i + \mathbf{r}^i)g(\mathbf{x}^i - \mathbf{r}^i)}}.$$

□

3.5 算法参数及选择依据

表 3: CS 算法参数

安全参数	含义	128	256	512
λ	安全等级	128	256	512
n	多项式环的维度	256	512	512
q	模数	32257	64513	64513
k, ℓ	环矩阵 $\mathbf{A}_0$ 的行列数	3,3	3,3	6,5
η	私钥的分布参数	1	1	1
a_0, b_0, B_0	$\mathbf{y}_0$ 相关参数	5,5,116	6,6,278	6,6,261
a_1, b_1, B_1	$\mathbf{y}_1$ 相关参数	10,9,2779	9,10,5385	12,11,12793
a_2, b_2, B_2	$\mathbf{y}_2$ 相关参数	6,9,1962	9,9,2569	8,11,9771
τ, τ'	挑战多项式相关参数	23,4	44,3	118,6
β, α	公钥压缩参数, hint 参数	32,2016	128,8064	32,4032
	c 的熵	131.08	256.4	512.18
M	签名重复次数	1.75	1.96	2.00

CS 算法的一个特点是参数多，这使得算法能通过优化参数配置，适配不同的应用场景。算法参数对算法各属性的影响如下：

1. 影响挑战多项式的熵的参数为 n, τ ;
2. 影响 MLWE 问题的困难性的主要参数有 n, q, ℓ, η ;
3. 影响 MSIS/BSTMSIS 的主要参数有 $n, q, k, B_i, \beta, \alpha$;
4. 影响签名重复次数的主要参数有 $n, k, \ell, a_i, b_i, B_i, \tau, \tau'$;
5. 影响算法尺寸的主要参数有 $n, k, \ell, a_i, b_i, B_i, \beta, \alpha$;

6. 影响算法效率的主要参数有 $n, q, k, \ell, a_i, b_i, B_i, \tau, \tau'$ 。

我们大致按照如下方式进行参数选择：

1. 选择 n, τ 使其熵达到目标安全强度 λ ，其中 n 为 2 的幂次；
2. 在保证 $2n|(q-1)$ 能使用 NTT 加速的前提下，选择 q, ℓ 使得 MLWE 问题满足安全要求，其中 q 是素数；
3. 选择 $k, a_i, b_i, B_i, \tau, \tau'$ 使得签名重复次数较小保证算法的效率且在 $\beta = 0, \alpha = 0$ 的情况下，MSIS/BSTMSIS 问题满足目标安全等级；若调整参数均无法满足条件，则需要返回第一步重新选择 n, q, ℓ 。
4. 选择 β, α 使得 MSIS/BSTMSIS 问题满足目标安全等级的情况下获得更紧致的签名算法；若调整参数均无法满足条件，则需要返回第三步重新选择参数。

上述流程可以帮助我们找到满足安全要求的算法参数，但要找到紧致且高效的参数组合仍然需要大量的实验和调优。我们在表 3 中给出了 CS 算法在不同安全等级下的参数选择。此外，还需要参数 M_i 来计算算法第 i 次迭代中第 9 步的拒绝概率， M_i 的计算公式见定理 3.2。

4 算法安全性

4.1 密码学定义

首先介绍格上的 MLWE 和 MSIS 困难问题。

定义 4.1 (判定 $\text{MLWE}_{n,q,k,\ell,\eta}$)。对于正整数 q, k, ℓ, η 以及 $\mathcal{R}$ 的维数 n ，我们称敌手 $\mathcal{A}_1$ 解决判定 $\text{MLWE}_{n,q,k,\ell,\eta}$ 问题的优势为

$$\text{Adv}_{n,q,k,\ell,\eta}^{\text{MLWE}}(\mathcal{A}_1) = \left| \frac{\Pr[b=1 | \mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}; \mathbf{b} \leftarrow \mathcal{R}_q^k; b \leftarrow \mathcal{A}_1(\mathbf{A}, \mathbf{b})]}{-\Pr[b=1 | \mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}; (\mathbf{s}_1, \mathbf{s}_2) \leftarrow S_\eta^\ell \times S_\eta^k; b \leftarrow \mathcal{A}_1(\mathbf{A}, \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2)]} \right|.$$

定义 4.2 ($\text{MSIS}_{n,q,k,\ell,\gamma}$)。对于正整数 q, k, ℓ ，正实数 γ 以及 $\mathcal{R}$ 的维数 n ，我们称敌手 $\mathcal{A}_2$ 解决 $\text{MSIS}_{n,q,k,\ell,\gamma}$ 问题的优势为

$$\text{Adv}_{n,q,k,\ell,\gamma}^{\text{MSIS}}(\mathcal{A}_2) = \Pr \left[\begin{array}{c} 0 < \|\mathbf{y}\|_\infty < \gamma \wedge \\ (\mathbf{A} | \mathbf{I}_k) \cdot \mathbf{y} = 0 \pmod{q} \end{array} \middle| \mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}; \mathbf{y} \in \mathcal{R}_q^{k+\ell} \leftarrow \mathcal{A}_2(\mathbf{A}) \right].$$

和其他双峰的 Fiat-Shamir 格签名算法一样，下面定义了 ML-DSA 中 SelfTargetMSIS 问题的一个变体，简记为 BSTMSIS 问题。然而与其他双峰的 Fiat-Shamir 格签名算法 BSTMSIS 问题不同点在于 CS 签名算法中的挑战多项式 c 的每一个系数均属于 $\{0, \pm 1\}$ 。

定义 5 ($\text{BSTMSIS}_{H,n,q,k,\ell,\gamma}$)。令 $H : \{0,1\}^* \times \mathcal{M} \rightarrow \mathcal{R}_2$ 为一个密码学哈希函数，其中 $M \subseteq \{0,1\}^*$ 是消息空间。令 $q, k, \ell > 0, \gamma \geq 0$ 以及 $\mathcal{R}$ 的维数 n 。在 (Q)ROM 下，一个多项式

时间的敌手 $\mathcal{A}_3$ 通过（量子）访问哈希函数 H 解决关于 $\mathbf{j} \in \mathcal{R}_2^k \setminus \{0\}$ 的搜索 $\text{BSTMSIS}_{H,n,q,k,\ell,\gamma}$ 问题的优势为

$$\text{Adv}_{H,n,q,k,\ell,\gamma}^{\text{BSTMSIS}}(\mathcal{A}_3) = \Pr \left[\begin{array}{l} 0 < \|\mathbf{y}\|_\infty < \gamma \wedge \\ H(\mathbf{A}\mathbf{y} - q\mathbf{c}\mathbf{j} \bmod 2q, \mu) = c \end{array} \middle| \begin{array}{l} (\mathbf{A}_0|\mathbf{b}) \leftarrow \mathcal{R}_q^{k \times \ell}; \\ \mathbf{A} = (2\mathbf{b} + q\mathbf{j}|2\mathbf{A}_0|2\mathbf{I}_k) \bmod 2q; \\ (\mathbf{y}, c, \mu) \in \mathcal{R}_q^{k+\ell} \times \mathcal{R}_2 \times \mathcal{M} \leftarrow \mathcal{A}_3^{|\mathcal{H}(\cdot)\rangle}(\mathbf{A}) \end{array} \right].$$

在经典 ROM 下，可以通过分叉引理完成从 MSIS 到 SelfTargetMSIS 的归约，该归约过程同样地可以应用到 BSTMSIS 问题。但由于涉及分叉引理，这两个归约过程都不能直接扩展到 QROM 归约中。需要注意的是，CS 签名算法使用的挑战多项式 c 的每一个系数均属于 $\{0, \pm 1\}$ ，为此，我们略微调整了传统的证明过程，在挑战多项式 c 的每一个系数均属于 $\{0, \pm 1\}$ 的情况下，完成了经典 ROM 下，MSIS 到 BSTMSIS 的归约过程。

引理 4.1 (从 MSIS 到 BSTMSIS 的经典归约). 令 $q > 0$ 为奇数模数, $H : \{0, 1\}^* \times \mathcal{M} \rightarrow \mathcal{R}_2$ 为一个被建模为随机预言机的密码学哈希函数, 并且假设每个多项式时间的经典算法对 $\text{MSIS}_{n,q,k,\ell,\gamma}$ 的优势都是可忽略的。那么每个多项式时间的经典算法对 $\text{BSTMSIS}_{n,q,k,\ell,\gamma/2}$ 的优势也是可忽略的。

证明. 考虑一个 $\text{BSTMSIS}_{n,q,k,\ell,\gamma/2}$ 经典算法 $\mathcal{A}$, 它是多项式时间的, 并且对 H 具有经典访问权限。如果 $\mathcal{A}^{H(\cdot)}(\mathbf{A})$ 对 $i = 1, \dots, Q$ 发起 Q 次哈希查询 $H(\mathbf{w}_i, \mu_i)$ 并对某个 $j \in [Q]$ 输出解 $(\mathbf{y}, c, \mu_j)$, 那么我们可以如下构造一个针对 $\text{MSIS}_{n,q,k,\ell,\gamma}$ 的敌手 $\mathcal{A}'$ 。

敌手 $\mathcal{A}'$ 首先将 $\mathcal{A}$ 回退到发起第 j 次查询的时刻, 并将哈希函数重新编程为 $H(\mathbf{w}_j, \mu_j) = c' (\neq c)$ 。然后, 以大约 $1/Q$ 的概率, 算法 $\mathcal{A}$ 将产生另一个解 $(\mathbf{y}', c', \mu_j)$ 。于是我们有

$$\begin{cases} \mathbf{A}\mathbf{y} - q\mathbf{c}\mathbf{j} = \mathbf{w}_j = \mathbf{A}\mathbf{y}' - q\mathbf{c}'\mathbf{j} \bmod 2q, \\ \|\mathbf{y}\|_\infty, \|\mathbf{y}'\|_\infty < \gamma/2. \end{cases}$$

由于 q 是奇数, 我们有 $\mathbf{A}(\mathbf{y} - \mathbf{y}') = (c - c')\mathbf{j} \bmod 2$ 。由于 $c' \neq c$, 结合 CS 算法中 c 的汉明重量为 τ 且熵至少为 $2^{-\lambda}$, 这意味着 $c' = c \bmod 2$ 的概率 $2^{-\lambda+\tau} \leq 2^{-\lambda/2}$ 是可忽略的, 因此, 在整数上 $\mathbf{y} - \mathbf{y}' = 0$ 的概率也是可忽略的。由于 $(\mathbf{b}|\mathbf{A}_0|\mathbf{I}_k) \cdot (\mathbf{y} - \mathbf{y}') = 0 \bmod q$ 并且 $\|\mathbf{y} - \mathbf{y}'\|_\infty < \gamma$, 它为矩阵 $(\mathbf{b}|\mathbf{A}_0|\mathbf{I}_k)$ 提供了一个 $\text{MSIS}_{n,q,k,\ell,\gamma}$ 解, 其中子矩阵 $(-\mathbf{b}|\mathbf{A}_0) \in \mathcal{R}_q^{k \times \ell}$ 是均匀分布。 $\square$

定义 4.3 (带中止的标准身份认证协议 (CID)). 带中止的标准身份认证协议 ID 是针对关系 R 的一个三轮交互协议, 在证明者 P 和验证者 V 之间进行。挑战由 $f(w)$ 生成, 设置为某个挑战集 C 上的分布 $U(C)$ 。证明者持有一对 $(x, y) \in R$, 而验证者只有 x , 其中对 (x, y) 由 PPT 算法 Gen 生成, 称为实例生成器。事件 “ $z = \perp$ ” 称为中止, 其概率 β 称为中止概率。

本文考虑的标准识别方案满足: 给定 c 和 z , 存在唯一的 w 使得 V 接受, 且 w 可以从 c 和 z 高效计算得到。

我们需要以下零知识变体, 称为完美接受诚实验证者零知识。

定义 4.4 (完美接受诚实验证者零知识). 设 ID 是一个标准识别方案。如果存在 PPT 模拟器 Sim , 使得对于任何由 (1^λ) 生成的 (x, y) , 交互 $\langle P(x, y) \leftrightarrow V(x) \rangle_{U(C)}$ 在条件 $z \neq \perp$ 下的输出分布 (w, c, z) 与 $\text{Sim}(x)$ 生成的 (w', c', z') 的分布完全相同, 则称其满足 paHVZK 。

为了避免两次使用相同的承诺，我们要求承诺的最小熵较高。

定义 4.5 (承诺最小熵). 对于 $\alpha \geq 0$ ，如果对于所有 $(x, y) \leftarrow \text{Gen}(1^\lambda)$ ，都有

$$H_\infty[w \mid (w, c, z) \leftarrow \langle P(x, y) \leftrightarrow V(x) \rangle_{U(C)}] \geq \alpha,$$

则称识别方案 ID (带有实例生成器 Gen) 具有承诺最小熵 α 。

最后，我们需要计算唯一响应的概念，以实现最终签名算法的强不可伪造性。

定义 4.6 (计算唯一响应). 设 ID 是一个带有实例生成器 Gen 的标准识别方案。对抗 ID 的 CUR 性质的敌手 $\mathcal{A}$ 的优势 $\text{Adv}_{\text{ID}}^{\text{CUR}}(\mathcal{A})$ 是其计算 (w, c, z, z') 的概率，使得 $V(x)$ 同时接受 (w, c, z) 和 (w, c, z') 且 $z \neq z'$ ，其中输入 x 由 $(x, y) \leftarrow \text{Gen}(1^\lambda)$ 生成。

定义 4.7 (数字签名算法). 数字签名算法是由 3 个概率多项式时间算法组成，包括：密钥生成算法 KeyGen 、签名生成算法 Sign 以及签名验证算法 Verify ，具体如下：

- $\text{KeyGen}(1^\lambda): 1^\lambda \rightarrow (\text{vk}, \text{sk})$ ，即输入安全参数 λ 输出验证密钥 vk 和签名密钥 sk ；
- $\text{Sign} : (\text{sk}, \mu) \rightarrow \sigma$ ，即以签名密钥 sk 和消息 μ 为输入，输出签名 σ ；
- $\text{Verify} : (\text{vk}, \mu, \sigma) \rightarrow b \in \{0, 1\}$ ，即以验证密钥 vk 、消息 μ 和签名 σ 为输入，输出拒绝或接受，是一个确定性算法。

令 $\gamma > 0$ ，如果对于 KeyGen 输出的任意密钥对 (vk, sk) 和任意消息 μ ，都有

$$\Pr[\text{Verify}(\text{vk}, \mu, \text{Sign}(\text{sk}, \mu)) = 1] \geq \gamma,$$

其概率取自签名算法的随机性，则称该签名算法是 γ -正确的。当概率还取自方案中使用的哈希函数所对应的随机预言机的随机性时，上述条件仍成立，则称该签名算法在 (Q)ROM 中是正确的。

下面是数字签名算法的安全性概念，即在无消息攻击下的存在性不可伪造性 (UF-NMA)、在选择消息攻击下的存在性不可伪造性 (UF-CMA) 以及在选择消息攻击下的强存在性不可伪造性 (sUF-CMA)。

定义 4.8 (签名算法的不可伪造性). 对于一个能访问量子随机预言机的多项式运行时间的量子敌手 $\mathcal{A}$ ，定义敌手 $\mathcal{A}$ 攻破签名算法 UF-NMA 安全属性的优势为：

$$\Pr_{(\text{vk}, \text{sk})} [\text{Verify}(\text{vk}, \mu^*, \sigma^*) = 1 \mid (\mu^*, \sigma^*) \leftarrow \mathcal{A}^H(\text{vk})] \leq \delta,$$

其中随机性取自 $\mathcal{A}$ 的随机硬币以及 $(\text{vk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$ 。若允许敌手访问签名预言机，但不能对消息 μ^* 进行签名询问时，则上述优势定义为敌手 $\mathcal{A}$ 攻破签名算法 UF-CMA 安全属性的优势。进一步地，若允许敌手对其消息 μ^* 进行签名询问但其输出的不是已查询过的签名时，上述优势定义为敌手 $\mathcal{A}$ 攻破签名算法 sUF-CMA 安全属性的优势。

4.2 签名算法安全性

和其他 Fiat-Shamir 类格签名算法一样，我们通过两步来证明 CS 签名算法的理论安全性。第一步我们通过引理 4.2 证明 CS 签名算法的 UF-NMA 安全性可以归约到 MLWE 问题以及 BSTMSIS 问题的困难性，后者在 ROM 下可以通过分叉引理归约到相应的 MSIS 问题。然后，构造能通

过 Fiat-Shamir 变换获得 CS 签名算法的 CS 身份认证协议, 并使用引理 4.3 证明 CS 身份认证协议具有如下性质: 承诺的极小熵至少为 n , 零知识性以及计算唯一回复属性。根据上述属性, 可以将 CS 签名算法的 (s)UF-CMA 安全性归约到 UF-NMA 安全性。最后结合上述两步, 定理 4.1 证明了 CS 签名算法的理论安全性。需要注意的是, 根据引理 B.3, CS 签名算法对拒绝概率 p_i 的近似最多减少签名算法 1 比特的安全性, 在这里我们忽略该影响, 认为 CS 签名算法的拒绝概率是 p_i 。

引理 4.2 (UF-NMA 安全性). 若存在一个敌手 $\mathcal{A}$ 能攻破 CS 签名算法的 UF-NMA 安全性, 则存在两个和敌手 $\mathcal{A}$ 相似运行时间的敌手 $\mathcal{B}$ 和敌手 $\mathcal{C}$ 使得:

$$Adv_{CS}^{UF-NMA}(\mathcal{A}) \leq Adv_{n,q,k,\ell,\eta}^{MLWE}(\mathcal{B}) + Adv_{H,n,q,k,\ell+1,B'_2}^{BSTMSIS}(\mathcal{C}).$$

证明. 为了描述方便, 我们假设公钥 $pk = (\mathbf{A}', \mathbf{b}) \in \mathcal{R}_q^{k \times \ell} \times \mathcal{R}_q^k$, 令 pk' 是从 $\mathcal{R}_q^{k \times \ell} \times \mathcal{R}_q^k$ 中均匀随机选取的, 称以 pk' 为公钥的 CS 签名算法变种为 CSU 签名算法。敌手 $\mathcal{B}$ 能调用敌手 $\mathcal{A}$, 敌手 $\mathcal{B}$ 根据敌手 $\mathcal{A}$ 对 CS 签名算法和 CSU 签名算法的不同表现来区分 pk 和 pk' 。具体来说, 若 $\mathcal{A}$ 给出了一个有效的伪造, 则认为该签名算法使用的公钥为 pk , 否则认为是 pk' , 即解决判定 $MLWE_{n,q,k,\ell,\eta}$ 问题。

敌手 $\mathcal{C}$ 则可通过敌手 $\mathcal{A}$ 的有效签名伪造来构造相应 BSTMSIS 的解。当 $\mathcal{A}$ 输出一个有效的伪造 $(\mu, \sigma = (\sigma_0, \sigma_1, \sigma_2, c))$ 时, 敌手 $\mathcal{C}$ 运行签名验证算法 12 来获得一个无穷范数小于 B'_2 的多项式向量 $\mathbf{z}' = (\mathbf{z}'_0, \mathbf{z}'_1, \mathbf{z}'_2)$ 、一个多项式 $\mathbf{w}'$ 以及一个多项式 w' , 且满足 $H(\mathbf{w}', w', \mu) = c$ 。进一步, 根据定理 3.1 知:

$$\alpha \mathbf{w}' + w' \mathbf{j} = \mathbf{A} \mathbf{z}' - qc \mathbf{j} \bmod 2q.$$

令 $H(\mathbf{w}', w', \mu) = H'(\alpha \mathbf{w}' + w' \mathbf{j}, \mu)$, 考虑到 Hash 函数的随机性, 忽略 H 与 H' 的区别, 则敌手 $\mathcal{C}$ 获得了一个 $BSTMSIS_{H,n,q,k,\ell+1,B'_2}$ 问题解 $(\mathbf{z}', c, \mu)$ 。□

相应的 CS 身份认证协议具体过程描述如下:

1. 证明者按照签名生成算法 11 生成 $\mathbf{y}$, 然后计算 $\mathbf{w} = \mathbf{A} \mathbf{y} \bmod 2q$, 计算 $\mathbf{w}^* = HighBits^h(\mathbf{w}, \alpha)$ 以及 $w^* = LSB(\mathbf{y}_0)$ 发送给验证者;
2. 验证者随机生成一个挑战多项式 c 发送给证明者;
3. 结合挑战多项式 c , 证明者按照签名生成算法 11 生成签名 σ 并发送给验证者;
4. 验证者执行签名验证算法 12 并获得 $\mathbf{z}' = (\mathbf{z}'_0, \mathbf{z}'_1, \mathbf{z}'_2)$; 若 $\|\mathbf{z}'_0\|_\infty \geq B'_0, \|\mathbf{z}'_1\|_\infty \geq B'_1, \|\mathbf{z}'_2\|_\infty \geq B'_2$ 且 $\mathbf{A} \mathbf{z}' - qc \mathbf{j} = \alpha \mathbf{w}^* + w^* \mathbf{j} \bmod 2q$, 则接受。

引理 4.3 (CS 身份认证协议属性). CS 身份认证协议满足如下性质:

1. CS 身份认证协议的承诺 $\mathbf{w}^*, w^*$ 的极小熵至少为 n ;
2. CS 身份认证协议满足 $paHVZK$ 性质;
3. 若敌手 $\mathcal{A}$ 能够攻破 CS 身份认证协议的计算唯一回复属性, 则存在敌手 $\mathcal{B}$ 和 $\mathcal{C}$ 在相似的运行时间下能够解决 $MLWE_{n,q,k,\ell,\eta}$ 问题和 $MSIS_{n,q,k,\ell+1,2B'_2}$ 问题, 即:

$$Adv_{CS}^{CUR}(\mathcal{A}) \leq Adv_{n,q,k,\ell,\eta}^{MLWE}(\mathcal{B}) + Adv_{n,q,k,\ell+1,2B'_2}^{MSIS}(\mathcal{C}).$$

证明. 首先证明极小熵, CS 身份认证协议的承诺为 $\mathbf{w}^*, w^*$, 其中 $w' = LSB(y_0)$ 的熵接近 n , 若加上承诺 $\mathbf{w}^*$, 则承诺的极小熵至少为 n .

下面证明 CS 身份认证协议的 paHVZK 性质. 在仅有公钥、 $\mathbf{b}_0$ 以及对 Hash 函数的控制能力下, 考虑如下的模拟算法:

1. 生成多项式向量 $\mathbf{z} = (\mathbf{z}_0, \mathbf{z}_1, \mathbf{z}_2)$, 其中 $\mathbf{z}_0 \leftarrow \overline{\mathcal{D}}_{a_0, b_0}^{B_0-N}, \mathbf{z}_1 \leftarrow \overline{\mathcal{D}}_{a_1, b_1}^{B_1-\tau}, \mathbf{z}_2 \leftarrow \overline{\mathcal{D}}_{a_2, b_2}^{B_2-\tau}$;
2. 随机生成挑战多项式 c , 并计算每次迭代中的挑战多项式 c'_i ;
3. 随机生成产生双峰的随机比特 bit_i , 并令 $c_r = \sum_{i=0}^{N-1} (-1)^{bit_i} c'_i$;
4. 计算 $\mathbf{w} = \mathbf{A}\mathbf{z} - qc\mathbf{j} - 2c_r\mathbf{b}_0 \bmod 2q$, 将承诺 $(\mathbf{w}^*, w^*) = (HighBits^h(\mathbf{w}, \alpha), LSB(\mathbf{z}_0 - c))$ 发送给验证者;
5. 令 $c = H(\mathbf{w}^*, w^*, \mu)$;
6. 计算 $\mathbf{h} = \mathbf{w}^* - HighBits^h(\mathbf{w} - 2\mathbf{z}_2 + 2c_r\mathbf{b}_0, \alpha) \bmod \pm \frac{2(q-1)}{\alpha}$;
7. 将签名 $\sigma = (\mathbf{z}_0, \mathbf{z}_1, \mathbf{h}, c)$ 发送给验证者。

模拟算法的第 1 步生成的 $\mathbf{z}$ 与签名算法 11 生成的同分布, 由于 $\mathbf{z}$ 是与 $bit_i, c, \mathbf{s}$ 均独立, 则给定 $\mathbf{z}$, 对于任意的 $bit_i, c_r, \mathbf{s}$, 总存在一个 $\mathbf{y}$ 使得 $\mathbf{z} = \mathbf{y} + c_r\mathbf{s}$. 所以模拟算法第 4 步生成的

$$\begin{aligned} \mathbf{w} &= \mathbf{A}\mathbf{z} - qc\mathbf{j} - 2c_r\mathbf{b}_0 \bmod 2q \\ &= \mathbf{A}(\mathbf{y} + c_r\mathbf{s}) - qc\mathbf{j} - 2c_r\mathbf{b}_0 \bmod 2q \\ &= \mathbf{A}\mathbf{y} \bmod 2q. \end{aligned}$$

此外, $LSB(\mathbf{z}_0 - c) = LSB(\mathbf{y}_0 + c_r - c) = LSB(\mathbf{y}_0)$, 因此, 上述模拟算法的 $\mathbf{w}, LSB(\mathbf{z}_0 - c)$ 分别完美模拟了签名算法中的 $\mathbf{A}\mathbf{y}, LSB(\mathbf{y}_0)$, 从而能完美模拟一个 (非拒绝情况下) 签名者. 因此, CS 身份认证协议满足 paHVZK 属性。

下面证明 CS 身份认证协议具有计算唯一回复的属性. 若 CS 身份认证协议不具有计算唯一回复属性, 则敌手 $\mathcal{A}$ 可以给出两个有效的身份认证协议通信脚本 $((\mathbf{w}^*, w^*), c, \sigma = (\mathbf{z}_0, \mathbf{z}_1, \mathbf{h}, c))$ 以及 $((\mathbf{w}^*, w^*), c, \sigma' = (\mathbf{z}'_0, \mathbf{z}'_1, \mathbf{h}', c))$. 敌手 $\mathcal{C}$ 根据身份认证的验证过程知: $\mathbf{A}\mathbf{z} - qc\mathbf{j} = \mathbf{A}\mathbf{z}' - qc\mathbf{j} \bmod 2q$, 即 $\mathbf{A}(\mathbf{z} - \mathbf{z}') = 0 \bmod 2q$, 即

$$(-\beta\mathbf{b}_1, \mathbf{A}', \mathbf{I}_k)(\mathbf{z} - \mathbf{z}') = 0 \bmod q.$$

若 $(\mathbf{z}_0, \mathbf{z}_1) = (\mathbf{z}'_0, \mathbf{z}'_1)$, 则 $\mathbf{h}' \neq \mathbf{h}$, $\mathbf{z}_2 \neq \mathbf{z}'_2$, 否则 $(\mathbf{z}_0, \mathbf{z}_1) \neq (\mathbf{z}'_0, \mathbf{z}'_1)$, 则 $\mathbf{z} \neq \mathbf{z}'$, 即敌手 $\mathcal{C}$ 找到了一个 $(-\beta\mathbf{b}_1, \mathbf{A}', \mathbf{I}_k)\mathbf{x} = 0 \bmod q$ 的解且 $\|\mathbf{x}\|_\infty \leq 2B'_2$.

然而 $(-\beta\mathbf{b}_1, \mathbf{A}')$ 可以弱化即一个 $LWE_{n,q,k,\ell,\eta}$ 实例即 $(-\mathbf{b}, \mathbf{A}')$. 令 CUS 身份认证协议是将 CS 身份认证协议中的 $(-\mathbf{b}, \mathbf{A}')$ 换成 $\mathcal{R}_q^k \times \mathcal{R}_q^{k \times \ell}$ 上的一个随机实例. 则敌手 $\mathcal{B}$ 将 CUS 或 CS 身份认证协议给敌手 $\mathcal{A}$, 若敌手 $\mathcal{A}$ 给出两个有效的身份认证协议通信脚本 $((\mathbf{w}^*, s^*), c, \sigma = (\mathbf{z}_0, \mathbf{z}_1, \mathbf{h}, c))$, 则敌手 $\mathcal{B}$ 判断输入的身份认证协议基于 LWE 实例 $(-\mathbf{b}, \mathbf{A}')$, 否则, 则认为基于 $\mathcal{R}_q^k \times \mathcal{R}_q^{k \times \ell}$ 上的随机元素, 从而解决判定 $MLWE_{n,q,k,\ell,\eta}$. $\square$

结合引理 4.2 以及引理 4.3, 定理 4.1 证明 CS 签名算法的 sUF-CMA 安全性可以归约到 MLWE、MSIS 以及 BSTMSIS 问题的困难性。

定理 4.1 (CS 签名算法的理论安全性). 假设 CS 身份认证协议的极小熵为 β_1 且输出为空的概率为 β_2 , 假设敌手 $\mathcal{A}$ 可以对 CS 签名算法进行 q_s 次签名询问以及 q_h 次 Hash 询问, 则存在敌手 $\mathcal{B}, \mathcal{C}$ 使得:

$$\begin{aligned} Adv_{CS}^{UF-CMA}(\mathcal{A}) &\leq Adv_{n,q,k,\ell,\eta}^{MLWE}(\mathcal{B}) + Adv_{H,n,q,k,\ell+1,B'_2}^{BSTMSIS}(\mathcal{C}) \\ &\quad + \frac{2^{-\beta_1/2+1}q_s}{1-\beta_2} \sqrt{q_h+1 + \frac{q_s}{1-\beta_2}} + 2^{-\beta_1/2+1}(q_h+1) \sqrt{\frac{q_s}{1-\beta_2}}. \end{aligned}$$

进一步地, 若 $\mathcal{A}$ 攻击 CS 签名算法的 sUF-CMA 安全性, 则有类似的结论:

$$\begin{aligned} Adv_{CS}^{sUF-CMA}(\mathcal{A}) &\leq 2Adv_{n,q,k,\ell,\eta}^{MLWE}(\mathcal{B}) + Adv_{H,n,q,k,\ell+1,B'_2}^{BSTMSIS}(\mathcal{C}) + Adv_{n,q,k,\ell+1,2B'_2}^{MSIS}(\mathcal{D}) \\ &\quad + \frac{2^{-\beta_1/2+1}q_s}{1-\beta_2} \sqrt{q_h+1 + \frac{q_s}{1-\beta_2}} + 2^{-\beta_1/2+1}(q_h+1) \sqrt{\frac{q_s}{1-\beta_2}}. \end{aligned}$$

4.3 实际安全强度

CS 签名算法的不可伪造性 (定理 4.1) 最终归约到 MLWE、MSIS 以及 BSTMSIS 三个困难问题。对于 MLWE 问题, 这里主要考虑两个最有效的攻击: 原始攻击 [BG14, AGV17] 和对偶攻击 [MR09, ADPS16, Alb17]。对于 MSIS 问题和 BSTMSIS 问题, 我们主要考虑无穷范数攻击, 更具体地, 像 HAETAE [CHE⁺24]、ML-DSA [LDK⁺22]、[Gar25] 一样, 将 $BSTMSIS_{H,n,q,k,\ell+1,B'_2}$ 问题当作 $MSIS_{H,n,q,k,\ell+1,B'_2}$ 问题来处理。此外, 在具体参数评估中, 我们把一个 $MLWE_{n,q,k,\ell,\eta}$ 实例看成维度为 $nk \times n\ell$ 的普通 LWE 实例; 把一个 $MSIS_{n,q,k,\ell,\gamma}$ 实例看成普通 SIS 实例。

BKZ 与 core-SVP 评估框架. 当前对 LWE/SIS 最好的攻击都基于块 BKZ 格基约化算法 [SE94, CN11, HPS11]。BKZ 算法用于解决 d 维格上的最短向量问题 (SVP), BKZ 算法通过迭代地运行 b 维筛法 (或枚举) 来解决更小的 b 维投影子格上的最短向量问题来找到 d 维格上的最短向量问题。其时间复杂度主要来源于筛法。在 Core-SVP 模型中, BKZ 算法被假设为只运行一次筛法。经典攻击下, 筛法 [BDGL⁺16, CL21] 的时间约为 $2^{0.292b}$, 量子攻击下, 筛法的时间约为 $2^{0.257b}$ 。

原始攻击. 原始攻击用于解决搜索 LWE 问题, 即恢复 LWE 样本中的秘密向量 $\mathbf{s}$ 。它把 LWE 的密钥恢复问题转化为在 q -ary 格中的短向量问题。给定 LWE 样本 $(\mathbf{A}, \mathbf{b} = \mathbf{As} + \mathbf{e}) \in \mathbb{Z}_q^{m \times l} \times \mathbb{Z}_q^m$, 构造格

$$\Lambda = \{(\mathbf{s}', \mathbf{e}', 1) \in \mathbb{Z}^{l+m+1} : \mathbf{As}' + \mathbf{e}' - \mathbf{b} = \mathbf{0} \bmod q\}.$$

因为存在短的 $(\mathbf{s}, \mathbf{e})$ 满足 $\mathbf{b} = \mathbf{As} + \mathbf{e} \bmod q$, 所以向量 $(\mathbf{s}, \mathbf{e}, 1)$ 是 Λ 中的一个短向量。原始攻击即在 Λ 上运行 BKZ 算法, 若找到了一个长度小于 $(\mathbf{s}, \mathbf{e}, 1)$ 的格向量, 则认为解决了搜索 LWE 问题。在实际评估中通过搜索 LWE 的样本个数 m 以及 BKZ 块大小 b 来找到最小的攻击代价。

对偶攻击. 对偶攻击用于解决判定 LWE 问题。给定 LWE 样本 $(\mathbf{A}, \mathbf{b})$ ，定义对偶格

$$\Lambda' = \{(\mathbf{u}, \mathbf{v}) \in \mathbb{Z}^m \times \mathbb{Z}^l : \mathbf{A}^\top \mathbf{u} + \mathbf{v} = \mathbf{0} \bmod q\}.$$

若 $(\mathbf{u}, \mathbf{v})$ 是 Λ' 的短向量，则

$$\mathbf{u}^\top \mathbf{b} = -\mathbf{v}^\top \mathbf{s} + \mathbf{u}^\top \mathbf{e}.$$

当 $\mathbf{b}$ 确实是 LWE 实例时，右端是两个小内积之和，因此 $\mathbf{u}^\top \mathbf{b}$ 在模 q 下取较小的值；当 $\mathbf{b}$ 均匀随机时， $\mathbf{u}^\top \mathbf{b}$ 在 $\mathbb{Z}_q$ 上均匀分布。攻击者收集足够多的短对偶向量后，即可通过统计检验区分 LWE 实例与均匀实例，从而解决判定 LWE 问题。在实际评估中通过搜索 LWE 的样本个数 m 以及 BKZ 块大小 b 来找到最小的攻击代价。

无穷范数攻击. 对无穷范数下的 SIS 问题，目前有效的攻击是无穷范数攻击 [LDK⁺22]。给定 $\mathbf{A} \in \mathbb{Z}_q^{m \times l}$ 与范数界 γ ，构造 q -ary 核格

$$\Lambda'' = \{\mathbf{u} \in \mathbb{Z}^{m+l} : \mathbf{B}\mathbf{u} = \mathbf{0} \bmod q\},$$

其中 $\mathbf{B} = (\mathbf{A} | \mathbf{I}_m)$ 。通过 BKZ 算法找到格 Λ'' 中的短非零向量即为 SIS 问题解。在实际评估中通过搜索 l 以及 BKZ 块大小 b 来找到最小的攻击代价。

在表 4 中我们给出了各安全等级下 $\text{MLWE}_{n,q,k,\ell,\eta}$ 在原始攻击和对偶攻击下的安全强度，以及 $\text{MSIS}_{n,q,k,\ell+1,B'_2}$ 、 $\text{MSIS}_{n,q,k,\ell+1,2B'_2}$ 在无穷范数攻击下的安全强度。

表 4: CS 签名算法实际安全强度

λ	算法安全参数	128	256	512
原始攻击	BKZ 块大小	450	978	1777
	经典安全强度	131.4	285.58	518.88
	量子安全强度	115.33	250.66	455.45
对偶攻击	BKZ 块大小	445	971	1766
	经典安全强度	129.94	283.5	515.67
	量子安全强度	114.05	248.87	452.63
无穷范数	BKZ 块大小	441(374)	916(797)	1879(1660)
	经典安全强度	128.81 (109.21)	267.47 (232.92)	548.67(484.72)
	量子安全强度	113.08 (95.86)	234.77 (204.47)	481.59(425.46)

4.4 其他安全特性

CS 算法的设计目标之一是实现全流程无浮点运算。掩码向量 $\mathbf{y}$ 通过中心多项分布 $\mathcal{D}_{a,b}$ 采样，仅涉及对短整数的均匀采样与加减运算；拒绝采样中，通过整数操作以接近 p_i 的概率接受签名。这些设计使得 CS 算法易于实现恒定时间 (constant-time) 操作，从而提升其抗侧信道攻击的能力。

5 算法性能与算法特点

5.1 算法性能

为评估 CS 算法在通用软件平台上的实际性能, 我们基于标准 C 语言实现了 CS-128、CS-256 和 CS-512 三个参数集, 并在统一的测试环境中对其运算效率与数据尺寸进行了测量。测试平台的详细配置如表 5 所示。

表 5: 软件实现平台配置

项目	配置
CPU	Intel Core i3-6100 @ 3.70GHz
内存	16GB
操作系统	Windows 7 64 位
编程语言	标准 C
编译器	Microsoft Visual Studio 2022 (MSVC)
指令集优化	无
密码库依赖	无

该实现未依赖任何专用密码学库, 也未使用 SIMD/AVX 等专用指令集加速或针对特定处理器进行汇编级优化, 因此可作为 CS 算法在通用 x86-64 平台上的参考实现性能。

运算效率。 表 2 给出了 CS 算法与 HAETAE、ML-DSA 在相同测试平台上密钥生成、签名和验证的效率对比 (单位: kcycles , 即千周期)。在 128 安全级别, CS-128 的密钥生成、签名和验证分别需要约 493、4442 和 632 千周期; 在 256 安全级别, CS-256 的相应操作分别需要约 763、14693 和 995 千周期; 在 512 安全级别, CS-512 的相应操作分别需要约 2343、48793 和 2962 千周期。实验结果表明, CS 算法的密钥生成效率优势明显, 验证效率也具有竞争力; 签名生成过程由于涉及迭代双峰拒绝采样及基于整数运算的拒绝概率近似计算, 耗时相对较长, 但仍优于同安全级别下 HAETAE 的签名生成效率。

传输与存储开销。 表 1 给出了 CS 算法与 HAETAE、ML-DSA 的公钥、签名和私钥尺寸对比 (单位: 字节)。CS-128 的公钥、签名和私钥尺寸分别为 976 B、1548 B 和 1888 B; CS-256 分别为 1760 B、3164 B 和 3968 B; CS-512 分别为 4288 B、5975 B 和 7808 B。CS 算法在公钥尺寸上具有明显优势, 在 128 和 256 安全级别均为三种方案中最小; 签名尺寸也较为紧致, 公钥与签名尺寸之和接近最小。

5.2 算法特点

创新性。 CS 算法的核心创新可概括为四点: (1) 使用双峰中心多项分布作为掩码分布, 其采样过程无浮点运算; 设计了基于整数运算的拒绝采样概率近似方法, 通过 Rényi 散度控制近似误差, 最多损失 1 比特安全强度; 这两个设计使得 CS 算法全程无浮点运算; (2) 引入灵活的迭代双峰参数 τ' , 在采样复杂度与拒绝概率计算复杂度之间取得更好的平衡; (3) CS 算法兼容有效

的公钥压缩机制，使得公钥更紧致。(4) 将传统双峰 FS 签名中挑战多项式的系数空间从 $\{0, 1\}$ 扩展到 $\{0, \pm 1\}$ ，在同等挑战熵下降低汉明重量，从而减小签名尺寸。

简洁性。 CS 算法保持了 Fiat-Shamir with Aborts 签名算法的经典三步结构：承诺 $\mathbf{w} = \mathbf{A}\mathbf{y}$ 、挑战 c 、响应 $\mathbf{z}$ 。其掩码采样仅需对短整数进行均匀采样和加减；核心多项式乘法基于 $\mathcal{R}_q$ 上 NTT 实现，模数 q 满足 $q \equiv 1 \pmod{2n}$ ，便于使用 NTT。拒绝采样中的概率计算主要通过整数的乘法以及右移操作来完成，无需浮点运算。总体来说，CS 算法全程无浮点运算都是通过简单的整数操作来完成，显著降低了实现复杂度和侧信道防护成本。

灵活性。 CS 算法在参数选择上可以根据算法使用场景的不同进行灵活调整。首先，底层困难问题的环维度 n 、矩阵维度 k, ℓ 可根据目标安全等级调整。其次，掩码向量 $\mathbf{y}$ 分三段，每段都可使用不同的中心多项分布，且中心多项分布有两个参数 a_i, b_i 比高斯分布更灵活。然后，参数 τ' 使得迭代双峰的设计更灵活：较小的 τ' 降低单次拒绝采样重复次数但增加迭代次数，较大的 τ' 则相反。再次，公钥压缩参数 β 和签名压缩参数 α 可在尺寸与安全性之间权衡。这种灵活的参数选择使 CS 能够通过优化参数配置，适配不同需求的应用场景（更高的安全性、更小的尺寸、更快的速度等）。

可扩展性。 CS 提供了 CS-128、CS-256 和 CS-512 三个参数集，分别对应约 128、256 和 512 比特的经典 core-SVP 安全强度（参见表 4）。由于 CS 算法的设计具有高度的参数灵活性可以通过参数调整来适应不同等级的安全需求。

参考文献

- [ADPS16] Erdem Alkim et al. “Post-quantum Key Exchange — A New Hope”. In: *Proceedings of the 25th USENIX Security Symposium (USENIX Security 2016)*. USENIX Association, 2016, pp. 327–343. URL: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/alkim>.
- [AGV17] Martin R. Albrecht et al. “Revisiting the Expected Cost of Solving uSVP and Applications to LWE”. In: *Advances in Cryptology – ASIACRYPT 2017*. Vol. 10624. LNCS. Springer, 2017, pp. 297–322.
- [Alb17] Martin R. Albrecht. “On Dual Lattice Attacks Against Small-Secret LWE and Parameter Choices in HELib and SEAL”. In: *Advances in Cryptology – EUROCRYPT 2017*. Vol. 10211. LNCS. Springer, 2017, pp. 103–129. DOI: [10.1007/978-3-319-56614-6_4](https://doi.org/10.1007/978-3-319-56614-6_4).
- [BDGL⁺16] Anja Becker et al. “New Directions in Nearest Neighbor Searching with Applications to Lattice Sieving”. In: *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2016)*. SIAM, 2016, pp. 10–24. DOI: [10.1137/1.9781611974331.ch2](https://doi.org/10.1137/1.9781611974331.ch2).

- [BG14] Shi Bai and Steven D. Galbraith. “An Improved Compression Technique for Signatures Based on Learning with Errors”. In: *Topics in Cryptology – CT-RSA 2014*. Vol. 8366. LNCS. Springer, 2014, pp. 28–47. DOI: [10.1007/978-3-319-04852-9_2](https://doi.org/10.1007/978-3-319-04852-9_2).
- [CHE⁺23] Jung Hee Cheon et al. *HAETAETAE: Algorithm Specifications and Supporting Documentation*. Tech. rep. Korea Internet & Security Agency, May 2023. URL: <https://www.haetae.org/>.
- [CHE⁺24] Jung Hee Cheon et al. “HAETAETAE: Shorter Lattice-Based Fiat-Shamir Signatures”. In: *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2024.3 (2024), pp. 25–75. DOI: [10.46586/tches.v2024.i3.25-75](https://doi.org/10.46586/tches.v2024.i3.25-75).
- [CL21] André Chailloux and Johanna Loyer. “Lattice Sieving via Quantum Random Walks”. In: *Advances in Cryptology – ASIACRYPT 2021, Part IV*. Vol. 13093. LNCS. Springer, 2021, pp. 63–91. DOI: [10.1007/978-3-030-92068-5_3](https://doi.org/10.1007/978-3-030-92068-5_3).
- [CN11] Yuanmi Chen and Phong Q. Nguyen. “BKZ 2.0: Better Lattice Security Estimates”. In: *Advances in Cryptology – ASIACRYPT 2011*. Vol. 7073. LNCS. Springer, 2011, pp. 1–20. DOI: [10.1007/978-3-642-25385-0_1](https://doi.org/10.1007/978-3-642-25385-0_1).
- [DDL13] Léo Ducas et al. “Lattice Signatures and Bimodal Gaussians”. In: *Advances in Cryptology – CRYPTO 2013, Part I*. Vol. 8042. LNCS. Springer, 2013, pp. 40–56. DOI: [10.1007/978-3-642-40041-4_3](https://doi.org/10.1007/978-3-642-40041-4_3).
- [Dud09] Jarek Duda. *Asymmetric Numeral Systems*. 2009. arXiv: [0902.0271](https://arxiv.org/abs/0902.0271). URL: <https://arxiv.org/abs/0902.0271>.
- [FIPS204] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. FIPS 204. National Institute of Standards and Technology, Aug. 2024. DOI: [10.6028/NIST.FIPS.204](https://doi.org/10.6028/NIST.FIPS.204). URL: <https://csrc.nist.gov/pubs/fips/204/final>.
- [Gar25] Joel Gärtner. “Compact Lattice Signatures via Iterative Rejection Sampling”. In: *Advances in Cryptology – CRYPTO 2025*. Vol. 16000. LNCS. Springer, 2025, pp. 514–547. DOI: [10.1007/978-3-032-01855-7_17](https://doi.org/10.1007/978-3-032-01855-7_17).
- [Gie14] Fabian Giesen. *Interleaved Entropy Coders*. 2014. arXiv: [1402.3392](https://arxiv.org/abs/1402.3392). URL: <https://arxiv.org/abs/1402.3392>.
- [HPS11] Guillaume Hanrot, Xavier Pujol, and Damien Stehlé. “Analyzing Blockwise Lattice Algorithms Using Dynamical Systems”. In: *Advances in Cryptology – CRYPTO 2011*. Vol. 6841. LNCS. Springer, 2011, pp. 447–464. DOI: [10.1007/978-3-642-22792-9_25](https://doi.org/10.1007/978-3-642-22792-9_25).
- [LDK⁺22] Vadim Lyubashevsky et al. *CRYSTALS-DILITHIUM*. Tech. rep. National Institute of Standards and Technology, 2022. URL: <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.

- [Lyu09] Vadim Lyubashevsky. “Fiat-Shamir with Aborts: Applications to Lattice and Factoring-Based Signatures”. In: *Advances in Cryptology – ASIACRYPT 2009*. Vol. 5912. LNCS. Springer, 2009, pp. 598–616. DOI: [10.1007/978-3-642-10366-7_35](https://doi.org/10.1007/978-3-642-10366-7_35).
- [Lyu12] Vadim Lyubashevsky. “Lattice Signatures without Trapdoors”. In: *Advances in Cryptology – EUROCRYPT 2012*. Vol. 7237. LNCS. Springer, 2012, pp. 738–755. DOI: [10.1007/978-3-642-29011-4_43](https://doi.org/10.1007/978-3-642-29011-4_43).
- [MR09] Daniele Micciancio and Oded Regev. “Lattice-based Cryptography”. In: *Post-quantum Cryptography*. Springer, 2009, pp. 147–191.
- [Pre17] Thomas Prest. “Sharper Bounds in Lattice-Based Cryptography Using the Rényi Divergence”. In: *Advances in Cryptology – ASIACRYPT 2017*. Vol. 10624. LNCS. Springer, 2017, pp. 347–374. DOI: [10.1007/978-3-319-70694-8_12](https://doi.org/10.1007/978-3-319-70694-8_12).
- [Sch91] Claus-Peter Schnorr. “Efficient Signature Generation by Smart Cards”. In: vol. 4. 3. Springer, 1991, pp. 161–174. DOI: [10.1007/BF00196725](https://doi.org/10.1007/BF00196725).
- [SE94] Claus-Peter Schnorr and Martin Euchner. “Lattice Basis Reduction: Improved Practical Algorithms and Solving Subset Sum Problems”. In: *Mathematical Programming* 66.1-3 (1994), pp. 181–199. DOI: [10.1007/BF01581144](https://doi.org/10.1007/BF01581144).
- [WKH⁺25] Joo Woo et al. *NTRU+Sign: Compact NTRU-Based Signatures Using Bimodal Distributions*. Cryptology ePrint Archive, Report 2025/106. 2025. URL: <https://eprint.iacr.org/2025/106>.

A 附录 A 中心多项分布的性质

该附录 A 主要介绍中心多项分布的一些性质。 $x \sim \mathcal{D}_{a,b}$ 的概率为 $\frac{f(x)}{2^{2ab}}$, 其中,

$$f_{a,b}(x) = \sum_{i=0}^{\lfloor \frac{x+a(2^b-1)}{2^b} \rfloor} (-1)^i \binom{2a}{i} \binom{x+a(2^b+1)-2^bi-1}{2a-1},$$

在不引起混淆的情况下, 将 $f_{a,b}(x)$ 简记为 $f(x)$ 。相应地, 对于一个 d 维独立向量 $\mathbf{x} \sim \mathcal{D}_{a,b}$, 其概率为 $\frac{g(\mathbf{x})}{2^{2abd}}$, 其中, $g(\mathbf{x}) = \prod_{i=0}^{d-1} f(\mathbf{x}_i)$ 。因此, $x \sim \mathcal{D}_{a,b,r}, \mathbf{x} \sim \mathcal{D}_{a,b,r}$ 的概率分别为

$$\frac{f(x-r)}{2^{2ab}}, \frac{g(\mathbf{x}-\mathbf{r})}{2^{2abd}}.$$

$x \sim \overline{\mathcal{D}}_{a,b,r}^t, \mathbf{x} \sim \overline{\mathcal{D}}_{a,b,r}^t$ 的概率为

$$\frac{f(x-r)}{\sum_{y=-t}^{y=t} f(y-r)}, \frac{g(\mathbf{x}-\mathbf{r})}{\prod_{i=0}^{d-1} \left(\sum_{y_i=-t}^{y_i=t} f(\mathbf{y}_i - \mathbf{r}_i) \right)}.$$

根据中心多项分布 $\mathcal{D}_{a,b}$ 的对称性知, $x \sim \overline{\mathcal{D}}_{a,b,\pm r}^t$ 的概率为

$$\frac{f(x-r) + f(x+r)}{\sum_{y=-t}^{y=t} f(y-r) + \sum_{y=-t}^{y=t} f(y+r)} = \frac{f(x-r) + f(x+r)}{2 \sum_{y=-t}^{y=t} f(y+r)}.$$

同样地根据中心多项分布 $\mathcal{D}_{a,b}$ 的对称性知, $\mathbf{x} \sim \overline{\mathcal{D}}_{a,b,\pm \mathbf{r}}^t$ 的概率为

$$\frac{g(\mathbf{x} - \mathbf{r}) + g(\mathbf{x} + \mathbf{r})}{\prod_{i=0}^{d-1} \left(\sum_{\mathbf{y}_i=-t}^{\mathbf{y}_i=t} f(\mathbf{y}_i - \mathbf{r}_i) \right) + \prod_{i=0}^{d-1} \left(\sum_{\mathbf{y}_i=-t}^{\mathbf{y}_i=t} f(\mathbf{y}_i + \mathbf{r}_i) \right)} = \frac{g(\mathbf{x} - \mathbf{r}) + g(\mathbf{x} + \mathbf{r})}{2 \prod_{i=0}^{d-1} \left(\sum_{\mathbf{y}_i=-t}^{\mathbf{y}_i=t} f(\mathbf{y}_i + \mathbf{r}_i) \right)}.$$

引理 A.1. 一维中心多项分布的概率质量函数 $f(x)$ 是对数凹函数, 对于任意的整数 a , $\frac{f^2(x)}{f(x+a)f(x-a)}$ 是一个关于 $|a|$ 的单调递增函数。

证明. 均匀分布是对数凹的, 且 $f(x)$ 是均匀分布的卷积, 因此 $f(x)$ 是对数凹的。令 $i > 0$, 根据对数凹函数的性质,

$$\frac{f(x)}{f(x+1)} \geq \frac{f(x-1)}{f(x)} \Leftrightarrow \frac{f(x+i)}{f(x+i+1)} \geq \frac{f(x-i-1)}{f(x-i)} \Leftrightarrow \frac{f(x-i)f(x+i)}{f(x-i-1)f(x+i+1)} \geq 1.$$

令 $h(a) = \frac{f^2(x)}{f(x+a)f(x-a)}$ 则,

$$h(|a|+1) = h(|a|) \frac{f(x+|a|)f(x-|a|)}{f(x-|a|-1)f(x+|a|+1)} \geq h(|a|).$$

□

B 附录 B 拒绝采样算法的相关引理

引理 B.1. 对于两个正整数 b, c , b', c' 相对于 b, c 的相对误差分别为 $\Delta_b = \frac{b'-b}{b}, \Delta_c = \frac{c'-c}{c}$, 则 $b' + c'$ 相对于 $b + c$ 的相对误差 Δ_{b+c} , 则 $b'c'$ 相对于 bc 的相对误差 Δ_{bc} , b'/c' 相对于 b/c 的相对误差 $\Delta_{b/c}$ 满足

$$\Delta_{b+c} = \frac{b\Delta_b + c\Delta_c}{b+c}, \Delta_{bc} = \Delta_b + \Delta_c + \Delta_b\Delta_c, \Delta_{b/c} = \frac{\Delta_b - \Delta_c}{1 + \Delta_c}.$$

证明. $b' + c'$ 相对与 $b + c$ 的相对误差

$$\Delta_{b+c} = \frac{b' + c'}{b + c} - 1 = \frac{b(1 + \Delta_b) + c(1 + \Delta_c)}{b + c} - 1 = \frac{b\Delta_b + c\Delta_c}{b + c}.$$

$b'c'$ 相对与 bc 的相对误差

$$\Delta_{bc} = \frac{b'c'}{bc} - 1 = \frac{b(1 + \Delta_b)c(1 + \Delta_c)}{bc} - 1 = \Delta_b + \Delta_c + \Delta_b\Delta_c.$$

b'/c' 相对与 b/c 的相对误差

$$\Delta_{b/c} = \frac{b'/c'}{b/c} - 1 = \frac{\Delta_b - \Delta_c}{1 + \Delta_c} = \frac{\Delta_b - \Delta_c}{1 + \Delta_c}.$$

□

引理 B.2. 令 $d = n(k + \ell + 1)$, 算法 4 输出 *False* 的概率 $\frac{N}{D}$ 与 p_i 的相对误差 Δ 满足 $\Delta \leq 2^{-L+4+\log d}$.

证明. 算法 4 输入两个 d 维向量 $\mathbf{z}, \mathbf{r}$, 该算法仅用整数运算完成了 p_i 的近似计算, 其误差主要来源于算法 5 的第 7 步和第 13 步的右移。首先计算算法 5 中单次右移的相对误差, 对于一个比特数大于 L 的整数 N , 单次右移 s 比特后获得一个长度等于 L 比特的整数 N_h 且满足

$$N = 2^s N_h + N_l$$

其中

$$N_l \in [0, 2^s).$$

单次右移的相对误差

$$\Delta' = \frac{N_l}{N} \leq \frac{2^s}{2^{s+L-1}} = 2^{1-L}.$$

算法 5 涉及到的右移次数最多为 $2d$, 根据引理 B.1 知 (在该算法中 $L = 64$, Δ' 很小, 我们简单假设 $\Delta_{bc} = \Delta_b + \Delta_c$): 算法 5 的相对误差 Δ_0 满足 $\Delta_0 \leq 2d\Delta' = 2^{-L+2+\log d}$.

令在算法 4 中 5 次调用算法 5 的相对误差分别为 $\Delta_1, \Delta_2, \Delta_3, \Delta_4, \Delta_5$, 则满足

$$|\Delta_1| \leq 2^{-L+3+\log d}, |\Delta_2| \leq 2^{-L+2+\log d}, |\Delta_3| \leq 2^{-L+2+\log d}, |\Delta_4| \leq 2^{-L+2+\log d}.$$

根据引理 B.1, 第 5 次调用算法 5 的相对误差 Δ_5 满足 (在这里, 每个 Δ_i 都小, 我们简单假设 $\Delta_{bc} = \Delta_b + \Delta_c$ 且忽略一次截断相对误差 Δ' 的影响, 同时忽略这里的 Approx 带来的 2^{1-L} 的误差)

$$|\Delta_5| \leq |\Delta_2| + \max(|\Delta_3|, |\Delta_4|).$$

因此, 根据引理 B.1, 算法 4 最终得到的 $\frac{N}{D}$ 相对于 p 的误差 Δ 满足 (在这里, 每个 Δ_i 都小, 简单假设 $\Delta_{b/c} = \Delta_b - \Delta_c$)

$$|\Delta| \leq |\Delta_1| + |\Delta_5| \leq |\Delta_1| + |\Delta_2| + \max(|\Delta_3|, |\Delta_4|) \leq 2^{-L+4+\log d}.$$

□

引理 B.3. 令 CSE 算法是将 CS 算法中 Reject 算法输出 False 的概率 $\frac{N}{D}$ 改为 p_i , 则若 CSE 算法在 $q_s = 2^{64}$ 次签名询问下的安全强度为 $\lambda + 1$ 比特, 则将 CS 算法至少有 λ 比特的安全强度。

证明. 根据引理 B.2, p_i 和 $\frac{N}{D}$ 的相对误差 Δ 满足 $|\Delta| \leq 2^{-L+4+\log d}$, 即 CSE 签名生成算法输出 $\mathbf{z}_N$ 的概率 $\prod_{i=1}^N p_{i-1}$ 与 CS 签名生成算法输出 $\mathbf{z}_N$ 的概率的相对误差 Δ_A 满足 $|\Delta_A| \leq N|\Delta|$ (在这里, 每个 Δ 都小, 简单假设 $\Delta_{bc} = \Delta_b + \Delta_c$), 在 CS 算法中 $N \leq 20, d \leq 6144, L = 64$, 因此, $|\Delta_A| \leq 2^{-43}$. 根据 [Pre17] 第 3.3 节中的结论知, 在 $q_s = 2^{64}$ 次签名询问且 $\lambda \leq 512$ 的情况下, 若 $|\Delta_A| \leq 2^{-38}$, 则 CS 算法中最多比 CSE 算法少 1 比特的实际安全强度。 □

引理 B.4. (与 [DDLL13] 的引理 2.1 类似)

沿用 2.7 小节的定义, 整数上的分布 $\mathcal{D}'_i, \mathcal{D}_i$ 的概率质量函数分别为 $g'_i(\mathbf{x}), g_i(\mathbf{x})$, 若存在常数 M_i 使得 $p_i = \frac{g_i(\mathbf{x})}{M_i g'_i(\mathbf{x})} \leq 1$, 则下面两个方式的输出分布是一致的,

1. $\mathbf{x} \leftarrow \mathcal{D}'_i$, 并以 p_i 的概率输出 $\mathbf{x}$;
2. $\mathbf{x} \leftarrow \mathcal{D}_i$, 并以 $\frac{1}{M_i}$ 的概率输出 $\mathbf{x}$ 。

证明. 根据2.7节的定义, C_i, C'_i 分别是 $\mathcal{D}_i, \mathcal{D}'_i$ 的定义域, 则第 1 种方式输出 $\mathbf{x}$ 的概率 p' 满足

$$g'_i(\mathbf{x}) \frac{g_i(\mathbf{x})}{M_i g'_i(\mathbf{x})} = \begin{cases} \frac{g_i(\mathbf{x})}{M_i}, & \mathbf{x} \in C_i \\ 0, & \mathbf{x} \in C'_i \setminus C_i \end{cases}$$

所以, 方式 1 输出为非空的概率为 $\sum_{\mathbf{x} \in C'_i} \frac{g_i(\mathbf{x})}{M_i} = \frac{1}{M_i}$ 与方式 2 的输出同分布。 $\square$